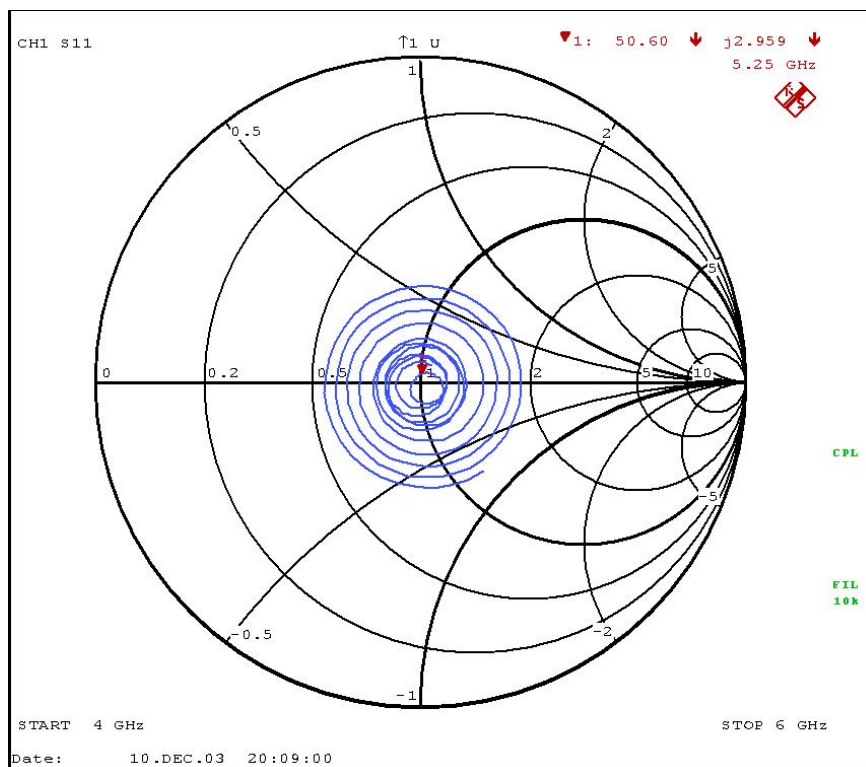


Patch-antenn



Johan Lönn
Ann-Sofie Nilsson
Jonas Olsson
David Svensson
Jonas Widén

Sammanfattning

Denna rapport är resultatet av ett projekt i kursen Systemutveckling, TNE067. Uppgiften bestod i att undersöka olika byggstrukturer lämpliga för 5GHz patch-antenn. Huvudmålet var att konstruera patch-antenn för 5,15-5,35 GHz och kunna utvärdera för och nackdelar relativt till varandra. Alla antenn ska vara matchade till 50Ω . Konstruktion av rektangulära $\lambda/4$ och $\lambda/2$ patch-antenn var huvudmålet och i mån om tid skulle även cirkulära patch-antenn konstrueras. De olika typerna av antenn ska sedan jämföras med varandra. Jämförelsen ska omfatta sådant som bandbredd, storlek på patchen och andra typiska antennparametrar. Obligatoriskt är också att producera en mönsterkortsprocessbar layout i Gerber-format.

Krav på färdig antenn sammanfattat: Bandbredd 200MHz vid en centerfrekvens på 5,25 GHz, det vill säga bandet 5,15-5,35 GHz ska täckas av en antenn. Först klara $SWR=3$, sedan jobba mot att $SWR=2$ ska täcka 5,15-5,35 GHz.

Det utvecklingsverktyg som har använts för att simulera patch-antennerna är Advanced System Design 2003.

Ett MatLab-program som beräknar parametrar för antennen, som till exempel längd och bredd, har även utvecklats.

Slutligen så tillverkades en $\lambda/2$, rektangulär antenn, som sedan uppmättes med hjälp av en vektoranalysator och gav ett lyckat resultat.

Innehållsförteckning

INLEDNING	1
SYFTE	1
METOD.....	1
TEORIBAKGRUND	2
OLIKA SÄTT ATT MATA EN PATCH-ANTENN.....	3
PATCH-ANTENNENS STRÅLNING.....	4
PATCH-ANTENN BERÄKNINGAR	6
MATLAB-PROGRAM	6
INTRESSANTA PARAMETRAR FÖR FRAMTAGNING AV PATCH-ANTENNER	6
PATCH-ANTENNENS LÄNGD.....	8
PATCH-ANTENNENS IMPEDANS OCH INSET LÄNGD	9
Inset-längd.....	10
FEED-LINE	11
CIRKULÄR PATCH-ANTENN	13
FUNKTION AV MATLAB-PROGRAM.....	14
BERÄKNINGAR MED MATLAB-PROGRAM.....	14
SIMULERING	15
$\lambda/2$ -ANTENN UTAN INSET FEED, LAYOUT	15
$\lambda/2$ -ANTENN UTAN INSET FEED, REFLEKTION.....	16
$\lambda/2$ -ANTENN UTAN INSET FEED, MATCHNING.....	16
$\lambda/2$ -ANTENN UTAN INSET FEED, VSWR.....	17
SLUTSATS $\lambda/2$ -ANTENN UTAN INSET FEED.....	17
$\lambda/2$ -ANTENN MED INSET FEED, LAYOUT	17
$\lambda/2$ -ANTENN MED INSET FEED, REFLEKTION.....	18
$\lambda/2$ -ANTENN MED INSET FEED, MATCHNING.....	18
$\lambda/2$ -ANTENN MED INSET FEED, VSWR	19
$\lambda/2$ -ANTENN MED INSET FEED, STRÖMMAR	19
$\lambda/2$ -ANTENN MED INSET FEED, E-FÄLT	20
SLUTSATS $\lambda/2$ -ANTENN MED INSET FEED.....	20
$\lambda/4$ -ANTENN MED INSET FEED, LAYOUT	21
$\lambda/4$ -ANTENN MED INSET FEED, REFLEKTION.....	21
$\lambda/4$ -ANTENN MED INSET FEED, MATCHNING.....	22
$\lambda/4$ -ANTENN MED INSET FEED, VSWR	22
$\lambda/4$ -ANTENN MED INSET FEED, STRÖMMAR	23
SLUTSATS $\lambda/4$ -ANTENN MED INSET FEED.....	24
$\lambda/2$ -ANTENN MED INSET FEED, 1.4 MM SUBSTRAT, REFLEKTION	24
$\lambda/2$ -ANTENN MED INSET FEED, 1.4 MM SUBSTRAT, MATCHNING	24
$\lambda/2$ -ANTENN MED INSET FEED, 1.4 MM SUBSTRAT, VSWR	25
RESULTAT/JÄMFÖRELSE SIMULERING	25
VERIFIERING	26
DESIGN AV PATCH-ANTENN	28
RESULTAT OCH SLUTSATS.....	29
REFERENSER.....	30
BILAGA 1 – MATLAB KOD	31

Inledning

Ett 4-poängs projektarbete har utförts i kursen Systemutveckling, TNE067. Projektet ska ge kunskap om hur en patch-antenn fungerar samt att ge en inblick i hur funktioner och hjälpmedel i utvecklingsverktyget ADS 2003 används.

Rapporten inleds med en allmän genomgång av patch-antenn. Sedan följer en beskrivning om de beräkningar som gjorts och det Matlab-program som utvecklats beskrivs lite närmare. Därefter visas resultaten av de simuleringar som gjorts. Avslutningsvis följer resultat och slutsats.

Syfte

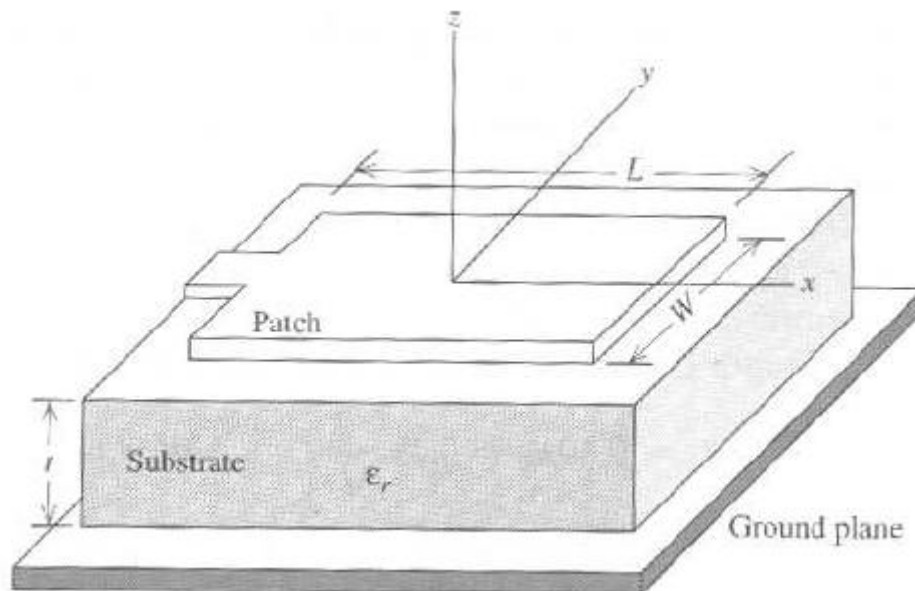
Syftet med denna rapport var att lära oss mer om patch-antenn, hur man kan designa och simulera med hjälp av ADS. Målet med detta projekt var också att kunna tillverka en fungerande antenn.

Metod

Inledningsvis utfördes en grundlig litteraturstudie. Gruppen delades sedan in i två grupper. Den ena gruppen koncentrerade sig på beräkningar av antennparametrar med hjälp av Matlab, medan den andra gruppen arbetade med simuleringar i ADS. Avslutningsvis valdes att tillverka ett mönsterkort för att verifiera de beräknade och simulerade resultaten.

Teoribakgrund

En patchantenn tillhör klassen resonanta antenner. För en rektangulär patch-antenn, som den i figur 1, gäller att den blir resonant när längden, L , är cirka halva våglängden av resonansfrekvensen. En patch-antenn är uppbyggd, ungefär som en kondensator, av två metall-lager med ett dielektriskt substrat emellan. Det ena metall-lagret är ett jordplan och det andra är själva patchen. Se figur 1.



Figur 1. Rektangulär patchantenn

På grund av patch-antenners enkla konstruktion är de relativt billiga att tillverka och massproducera, vilket gör dem mycket populära. Dom är även lätta att integrera samman med annan elektronik och kan ofta konstrueras på samma mönsterkort, vilket sparar utrymme. Andra fördelar är bland annat att de är små, platta och bara väger några gram. Längden på en rektangulär patch bestäms av våglängden, λ . Som ett exempel kan nämnas att för en $\lambda/2$, rektangulär patch med resonansfrekvensen 5 GHz ligger längden, L , runt 13 mm och vidden, W , ligger i området $L - 2L$. Dom största nackdelarna med patch-antennerna är dålig bandbredd, stora förluster, låg förstärkning och att de bara klarar en effekt upp till 100 W.¹

Det finns många utföranden på själva patchen. Det vanligaste är dock att ha enkla geometriska former som till exempel rektangulära, cirkulära och triangulära på grund av att det är lättare att göra beräkningar på dessa typer. Annars kan patchen se ut på många olika sätt. Det går även att koppla ihop flera patchar med varandra till en så kallad patch-array. Patcharna binds samman med microstripledare. En patch-array har betydligt bättre bandbredd och effekt än en vanlig enkel patch-antenn. Symmetriska patchar ger en linjär polarisation och osymmetriska patchar ger cirkulär polarisation. För att erhålla cirkulär polarisation hos en rektangulär patch-antenn kan exempelvis två diagonala hörn trunkeras, vilket gör att patchen får en osymmetrisk form. Ett annat vanligt sätt att ändra en patch-antenns egenskaper är att skära ut olika slitsar och dylikt i patchen. Genom att konstruera en patch-antenn med två ingångar så kan dualband erhållas, det vill säga antennen kan arbeta vid två olika resonansfrekvenser.²

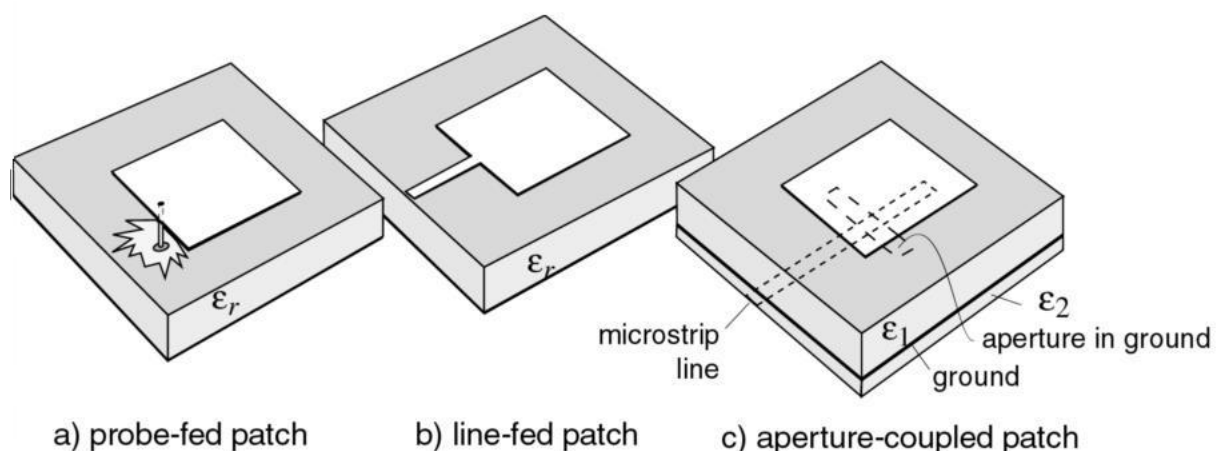
¹ Se referens [1] och [3]

² Se referens [1]

Patch-antennor lämpar sig bra för olika applikationer som inte kräver så mycket effekt. Arrayer av microstripantennor som klarar en någorlunda hög effekt används ofta till radar och liknande. Patch-antennor kan användas för frekvenser mellan 100 MHz och 100 GHz. För frekvenser i gigahertzområdet är dock räckvidden inte speciellt stor. Strålningen dämpas också kraftigt när den passerar genom tjocka väggar, vilket gör att de lämpar sig bäst för inomhusbruk. Ett typiskt användningsområde är WLAN- och Bluetooth-applikationer. Patch-antennor används också till mobilkommunikationsutrustning, där små och billiga antennor fordras.³

Olika sätt att mata en patch-antenn

Patch-antennor kan matas på olika sätt för att få olika egenskaper. De tre vanligaste är koaxial-matning, microstrip och aperture-coupled. Vid matning med koaxial-prob låter man proben komma upp underifrån genom jordplanet och substratet. Koaxial-matning används oftast när substratet är väldigt tunt och ger en väldigt liten bandbredd. En fördel är att proben kan placeras på valfri plats på patchen för att uppnå matchning för önskad impedans, då impedansen varierar på patchen. Matning med microstripledare är väldigt praktiskt då patch-antennen kan etsas på samma substrat som tillhörande kretsar i designen, som till exempel förstärkare och filter och man slipper att göra hål i substratet. Det här reducerar tillverkningskostnaden, men bandbredden blir väldigt liten även här. Impedansen i kanten på en rektangulär patch ligger i området 150-300 ohm. För att uppnå matchning för en annan impedans brukar en slits göras i patchen och microstripledaren skjuts in mot mitten av patchen, så kallad inset feed, mer om det beskrivs senare i rapporten.



Figur 2. Olika matningsätt för patchantenn

Ett problem med ovanstående matningssätt är att oönskad strålning fås från matningsledaren, vilket resulterar i att patch-antennens egenskaper kan försämrats. En lösning på detta är att använda sig av matningssättet aperture-coupled. Detta innebär kort beskrivet att ett extra substratlager läggs under jordplanet och microstripledaren ligger på det lagret istället. Sedan görs en slits i jordplanet så att det blir en elektromagnetisk kontakt mellan microstripledaren och patchen. Genom att ändra utformning och storlek på slitsen i jordplanet kan bättre bandbredd för patch-antennen erhållas.⁴

³ Se referens [2] och [3]

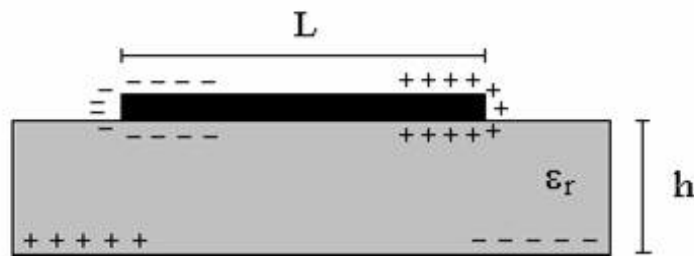
⁴

Valet av substrat är väldigt viktigt när patch-antennerna ska konstrueras. De bästa antennegenskaperna fås då substratet är tjockt och permitiviteten, ϵ_r är låg. Detta är tvärtom vad som önskas i vanliga fall när andra applikationer ska göras på mönsterkort, då ett tunt substrat med hög permitivitet är att föredra. När patchen etsas på samma substrat som övriga kretsar i applikationen måste en kompromiss göras när det gäller val av substrat. Oftast väljs någonting mitt emellan. En stor fördel är då att använda matningssättet aperture-coupled, då skilda substrat används för patch-antennen och resten av konstruktionen. Ett tjockt substrat med lågt ϵ_r kan då användas för antennen, vilket innebär att det inte blir så stora impedansförluster i patchen, samt att en bättre bandbredd erhålls. Nackdelen är att detta är en dyrare konstruktion.⁵

Patch-antennens strålning

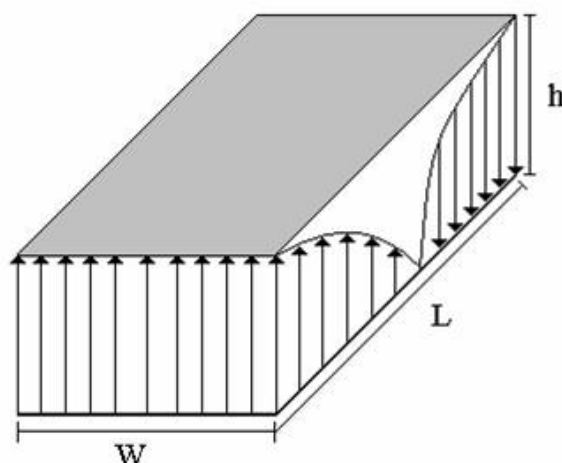
Strålningen från en rektangulär patch-antenn kan ses som ett halvklot som strålar ut från patchens ovansida.

När patchen matas på ena kortsidan uppstår en laddningsfördelning som man kan se i figur 3.



Figur 3. Laddningsfördelning i substrat

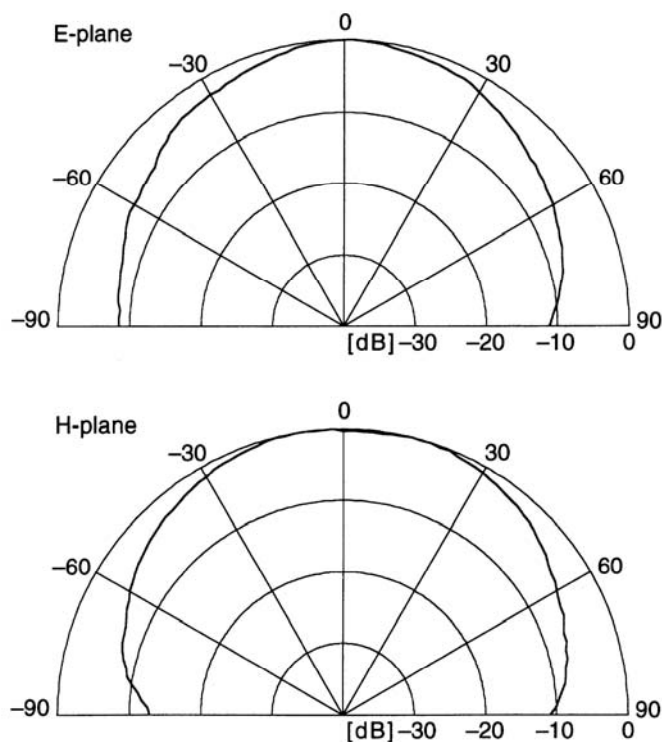
Då uppstår ett elektriskt fält med följande utseende.



Figur 4. Elektriska fält i substrat

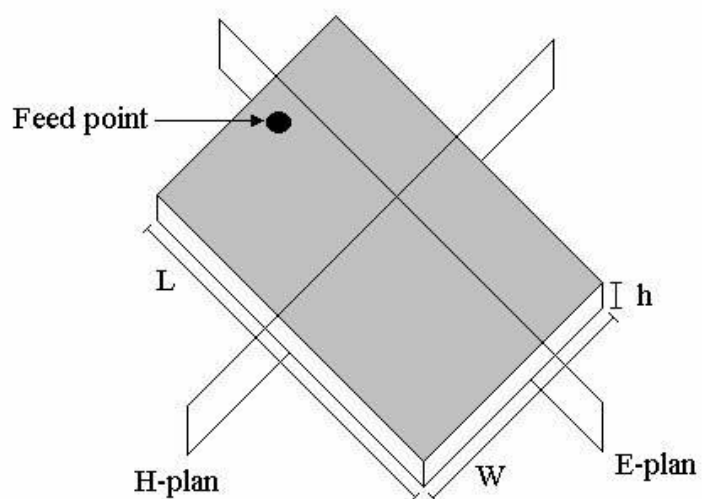
⁵ Se referens [1]

Detta skapar ett strålningsfält som är halvklotsformat på patchens ovansida. I diagrammen i figur 5 för E-planet och H-planet ser man tydligt halvklotsformen på strålningen.



Figur 5. E-plan och H-plan för en rektangulär patch

Placeringen på E-planet och H-planet visas i figur 6.⁶



Figur 6. Position för E-plan och H-plan

⁶ Se referens [1] och [2]

Patch-antenn beräkningar

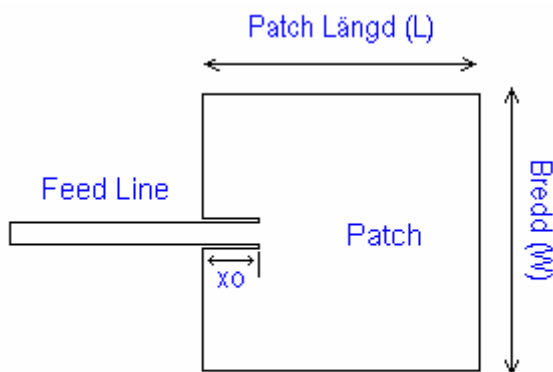
Detta avsnitt av rapporten kommer att beröra de beräkningsalgoritmer som används vid framtagandet av ett MatLab program för parameter framtagning för patch-antenn.

MatLab-program

MatLab-programmet utvecklades för att dels underlätta vid simuleringen av patch-antenn, dels för att öka förståelsen för hur de olika parametrarna påverkar patch-antennen. MatLab-programmet kan på ett snabbt sätt generera de olika parametrar som behövs för att konstruera en antenn. Simuleringar gjordes med hjälp av ADS2003, som är ett kraftfullt simuleringsverktyg för bland annat patch-antenn.

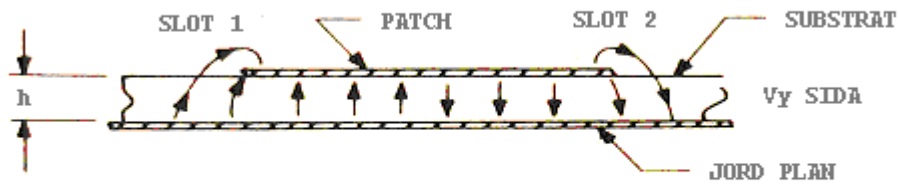
De algoritmer som används för att beräkna resonansfrekvens, patch-antennens längd och bredd för att nämna några parametrar, bygger på approximationer vilka har härletts från olika författare inom området patch-antenn. Mycket information har hämtats från IEEE som är en bra informationskälla för detta område. I och med att algoritmerna är approximativa finns en viss tolerans på de beräknade värdena, vilka kan sträcka sig till $\pm 3\%$, kanske mer för vissa parametrar. Programmet är inte gjort för att utföra exakta beräkningar, utan för att visa riktvärden för simuleringarna.

Intressanta parametrar för framtagning av patch-antenn.



Figur 7. Parametrar för patch-antenn

Patch-antennens längd (L) är den parameter som bestämmer arbetsfrekvensen på antennen. När beräkningar utförs på patch-antennlängden är det viktigt att tänka på, de elektriska fältlinjer som genereras vid applicering av en signal på Feed Line inte kommer att ha en 90 graders vinkel mot substratet och jordplanet, utan får en böjd form. Detta kommer att påverka patch-antennens längd och kallas fringing faktor.



Figur 8. Patch-antenn från sidan som visar fringing faktor

Som figur 2 visar går fältlinjerna i en böjd bana som gör att den fysiska patch antenn längden blir längre. Slot 1 och slot 2 är namn för patch-antennens motsatta långsidor vilket representerar antennens bredd. Bredden på patch-antennen är inte lika strikt bunden till antennens resonansfrekvens som längden, utan påverkar resistansen vid resonans. Genom att öka bredden kommer även resistansen att minska och vice versa. Resistansen vid Slot 1 och Slot 2 kan variera från cirka 150 till 300 ohm beroende på substratets dielektriska egenskaper. För att en matchning vid 50 ohm ska kunna tillämpas används en stripline matchad till 50 ohm. Denna skjuts in till en punkt på patch-antennen där resistansen är 50 ohm (xo se fig1). Detta kan uppnås eftersom resistansen på patch-antennen minskar in till patch-antennens mitt där resistansen är 0 ohm.

Patchen kan ses som en transmissionsledning. För att se hur sambanden för resistansen kan tecknas, kan en modell av patch antenn tas fram i form av en transmissionslednings modell. Ur den kan sedan admittansen för vare slot tas fram.

Patch-antennens längd

För att beräkna resonansfrekvensen på patch-antennen för halv- och kvartsvåg användes formeln⁷

$$f_r = \frac{1}{2\sqrt{\epsilon_o \mu_o} (L + 2\Delta L) \sqrt{\epsilon_{eff}}} \quad (1)$$

Där ϵ_o är Permeabilitet i vakuum, μ_o är permeabilitet i vakuum och Fringing-faktorn (ΔL).

Effektiv dielektrisk konstant (ϵ_{eff}) beräknas enligt formel⁸.

$$\epsilon_{eff} = \left(\frac{\epsilon_r + 1}{2} + \frac{\epsilon_r - 1}{2} \right) \sqrt{1 + \frac{10h}{W}} \quad (2)$$

Där ϵ_r är relativ permitivitet för substratet, W representerar patch-antennens bredd och h är höjden på substratet.

Fringing-faktorn beräknas enligt formeln⁹.

$$\Delta L = 0.412h \left[\frac{\epsilon_{eff} + 0.3}{\epsilon_{eff} - 0.258} \right] \left[\frac{W/h + 0.264}{W/h + 0.813} \right] \quad (3)$$

För att patch-antennens längd och resonansfrekvens ska bli så optimal som möjligt med formel (1) används en praxis att L ska befinna sig i intervallet $L = 0.48\lambda_g \sim 0.49\lambda_g$.

Detta uppnås genom att variera L inom intervallet och från detta välja det värde på L som ger den bäst matchade resonansfrekvensen, som fås av formel (1). Av formeln nedan fås λ_g .

$$\lambda_g = \frac{c}{f_r \sqrt{\epsilon_r}} \quad (4)$$

Där c är ljusets hastighet i vakuum.

Den erhållna längden som fås genom beräkningar av ovanstående formler används till halv- och kvartsvågs-antenn. För kvartsvågs-antenn används halva längden.

⁷ Se referens [4]

⁸ Se referens [5]

⁹ Se referens [6]

Patch-antennens bredd

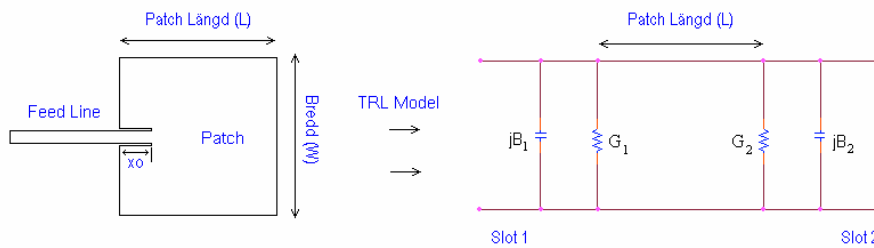
Patch-antennens bredd beräknas enligt formeln¹⁰.

$$W = \frac{1}{2f_r \sqrt{\mu_0 \epsilon_0}} \sqrt{\frac{2}{\epsilon_r + 1}} \quad (5)$$

Vid beräkningar av patch-antennens bredd bör bredden hamna inom intervallet $L < W < 2L$ för fullgod funktion.

Patch-antennens impedans och Inset längd

Vid beräkning av patch-antennens admittans i kanterna (i Slot 1 och Slot 2) kan patch-antennen göras om till en förenklad transmissionsledningsmodell.



Figur 9. Transmissionsledningsmodell av patch-antenn

Den komplexa admittansen vid varje slot består av en realdel, G_n och en imaginärdel, B_n . Vid resonansfrekvensen kommer den imaginära delen att bli 0, vilket medför att endast två konduktanser kvarstår G_1 och G_2 , utifrån dessa kan formlerna^{11 12} härledas, då Slot 1 och Slot 2 är identiska.

$$Y_{in} = Y_1 + Y_2 = 2G_1 \quad (6)$$

$$Z_{in} = \frac{1}{Y_{in}} = \frac{1}{2G_1} \quad (7)$$

Genom att använda den gemensamma konduktansen G_{12} kan även kantresistansen tecknas med hjälp av formeln.

$$R_{in} = \frac{1}{2(G_1 + G_{12})} \quad (8)$$

¹⁰ Se referens [4]

¹¹ Se referens [4]

¹² Se referens [7]

Konduktanserna G_1 och G_2 kan beräknas genom formeln¹³.

$$G_n = \frac{\pi W}{\lambda_o Z_f} [1 + (1 - (0.636 \ln k_o W_b))] \quad (9)$$

Här används grund våglängden λ_o ,

$$\lambda_o = \frac{c}{fr} \quad (9b)$$

$$Z_f = \sqrt{\frac{\mu_o}{\epsilon_o}} \quad (9c)$$

$$k_o = \frac{2\pi fr}{c}, \quad W_b \approx th \quad (9d)$$

Den gemensamma konduktansen G_{12} kan beräknas genom att använda sig av uttrycket nedan¹⁴.

$$G_{12} = \frac{1}{120\pi^2} \int_0^\pi \left[\frac{\left(\frac{\sin(k_o W)}{2} \cos(\Theta) \right)}{\cos(\Theta)} \right]^2 J_0(k_o L \sin(\Theta)) \sin^3(\Theta) d\Theta \quad (10)$$

Inset-längd

För att patch-antennen ska fungera optimalt måste den matchas med sina kringkomponenter. Detta kan göras genom att fasa in en stripledare på patch-antennen så att matchning uppnås. I detta fall ska 50 ohms matchning uppnås. Eftersom resistansen utbreder sig som en kvadratisk cosinus-funktion längs patch-antennens längd kan Inset-längden beräknas med formeln [4].

$$R_{in}(x = x_0) = \frac{1}{2(G_1 + G_{12})} \cos^2\left(\frac{\pi \cdot x_0}{L}\right) \quad (11)$$

R_{in} är kantresistansen då $x_0=0$ genom att sätta $R_{in}= 50$ ohm och lösa ut x_0 fås Inset längden.

¹³ Se referens [8]

¹⁴ Se referens [7]

Feed-Line

För att koppla in antennen till systemet så har en microstrip använts. Ledaren matchades för 50Ω . Följande formler har använts¹⁵:

$$Z_f = \sqrt{\frac{\mu_0}{\epsilon_0}} \quad (12)$$

För att veta hur bred microstripen skall vara, används följande formler:

$$A = 2\pi \frac{Z_0}{Z_f} \sqrt{\frac{\epsilon_r + 1}{2}} + \frac{\epsilon_r - 1}{\epsilon_r + 1} \left(0.23 + \frac{0.11}{\epsilon_r} \right) \quad (13)$$

$$w = h \frac{8e^A}{e^{2A} - 2} \quad (14)$$

Om $\frac{w}{h} \geq 2$, så måste w räknas om med denna formel:

$$w = \frac{2h}{\pi} \left(B - 1 - \ln(2B - 1) + \frac{\epsilon_r - 1}{2\epsilon_r} \left[\ln(B - 1) + 0.39 - \frac{0.61}{\epsilon_r} \right] \right) \quad (15)$$

$$\text{där } B = \frac{Z_f \pi}{2Z_0 \sqrt{\epsilon_r}} \quad (16)$$

Genom att använda ekvationen nedan fås längden av microstripen:

$$L = \frac{c}{\sqrt{\epsilon_{eff}} * f_r} \quad (17)$$

Där c är ljusets hastighet, f_r är den önskade frekvensen och ϵ_{eff} är den effektiva dielektriska konstanten vilken beräknas med följande formler:

Vid ett förhållande där $\frac{w}{h} > 1$ så används:

$$\epsilon_{eff} = \frac{\epsilon_r + 1}{2} + \frac{\epsilon_r - 1}{2} \left(1 + 12 \frac{h}{w} \right)^{-0.5} \quad (18)$$

¹⁵ Se referens [10]

Alternativt då $\frac{w}{h} < 1$:

$$\varepsilon_{eff} = \frac{\varepsilon_r + 1}{2} + \frac{\varepsilon_r - 1}{2} \left[\left(1 + 12 \frac{h}{w} \right)^{-0.5} + 0.04 \left(1 - \frac{w}{h} \right)^2 \right] \quad (19)$$

För att kontrollera sina värden kan följande formel användas:

$$Z_0 = \frac{Z_f}{\sqrt{\varepsilon_{eff} \left(1.393 + \frac{w}{h} + \frac{2}{3} \ln \left(\frac{w}{h} + 1.444 \right) \right)}} \quad (20)$$

Z_0 bör då vara så nära den önskade impedansen som möjligt. Om Z_0 inte stämmer så medför detta att antennen inte är matchad med resten av systemet.

Cirkulär Patch-antenn

Då en patch-antenn är elektriskt större till ytan, än fysiskt måste detta tas hänsyn till. Detta kan göras på många sätt och med olika noggrannhet, i formlerna nedan visas en enklare modell¹⁶.

$$r = \frac{c * k_{10}}{2\pi * f_r \sqrt{\epsilon_r}}$$

Där k_{10} är 1.84118 för den mest dominanta resonansfrekvensen

$$\lambda_g = \frac{c\sqrt{\epsilon_r}}{f_r} \quad \lambda_0 = 3.413 * r$$

$$\Delta_\lambda = 1 + \text{real}\left(1 - \frac{\lambda_g}{\lambda_0}\right)$$

$$r_{true} = \frac{c * k_{10}}{2\pi * r * \Delta_\lambda * \epsilon_r}$$

För att kontrollera dessa värden kan denna ekvation användas:

$$Circular_{fr} = \frac{c * k_{10}}{2\pi * r_{true} * \Delta_\lambda * \sqrt{\epsilon_r}}$$

Om denna inte stämmer med den önskade frekvensen så har antennen fel radie. Även den cirkulära antennen måste matchas för att uppnå maximal förstärkning. Detta kan göras på flera sätt, ett är att göra enligt samma princip som den rektangulära. Hur man beräknar detta kommer inte att tas upp i rapporten.

¹⁶ Se referens [11]

Funktion av MatLab-program

MatLab-programmet är uppbyggt kring ett grafiskt gränssnitt vilket gör att det är överskådligare och enklare att hantera.

De parametrar som användaren kan använda sig av är höjden på substratet, relativ permitivitet för substratet och resonansfrekvensen. Programmet räknar sedan fram approximativa värden för $\lambda/2$, $\lambda/4$ – rektangulära och cirkulära patch-antennar. Med lämpliga stripledningsparametrar.

Beräkningar med MatLab-program

Först matas höjden och permitivitet för substratet in samt den resonansfrekvens som önskas. Tryck på knappen Calc och parametrarna uppdateras med beräknade värden. Reset knappen nollställer alla uträknade parametrar.

MicroStrip_PatchV3.1

Rectangular Patch Antenna Half Wavelength

Custom value

Substrate hight (mm)

Permittivity

Patch Frequency (Hz)

Patch Antenna Characteristics

Patch Length (mm)

Patch Width (mm)

Patch Frequency (Hz)

Patch Edge Imp (ohm)

Inset Feed Point (mm)

Inset Feed Point Imp (ohm)

Patch Antenna Feed Line

Feed Line Length (mm)

Feed Line Width (mm)

Feed Line Imp check (ohm)

Rectangular Patch Antenna Quarter Wavelength

Custom value

Substrate hight (mm)

Permittivity

Patch Frequency (Hz)

Patch Antenna Characteristics

Patch Length (mm)

Patch Width (mm)

Patch Frequency (Hz)

Patch Edge Imp (ohm)

Inset Feed Point (mm)

Inset Feed Point Imp (ohm)

Patch Antenna Feed Line

Feed Line Length (mm)

Feed Line Width (mm)

Feed Line Frequency (Hz)

Circular Patch Antenna

Custom value

Substrate hight (mm)

Permittivity

Patch Frequency (Hz)

Patch Antenna Characteristics

Patch Radie (mm)

Patch Frequency (Hz)

Patch Edge Imp (ohm)

Inset Feed Point (mm)

Inset Feed Point Imp (ohm)

Patch Antenna Feed Line

Feed Line Length (mm)

Feed Line Width (mm)

Feed Line Frequency (Hz)

Calculation program of Patch Antenna.
The program was made in the course TNE067
Systemutveckling at Linköpings University 2003
to calculate approx values for patch antenna.

Calc Reset

Figur 10. MatLab-program för beräkning av patch-antennar

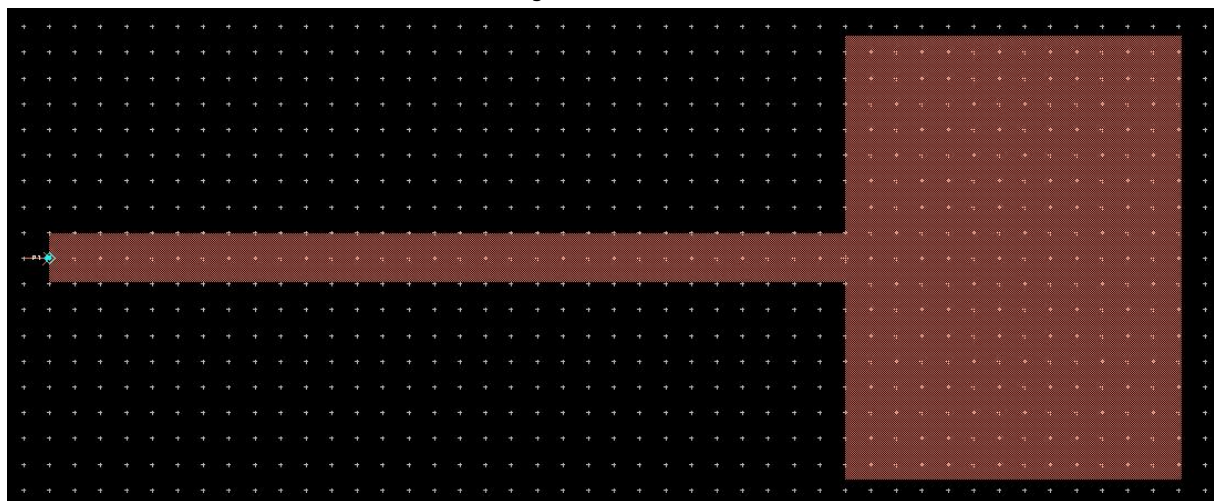
Simulering

Simulering av patch-antennerna har utförts i programmet *Advanced Design System 2003A*. I programmet har simuleringsverktyget *Momentum* varit till stor hjälp vid analysering av simuleringarna.

Gruppen har inriktat sig på att simulera $\lambda/2$ - samt $\lambda/4$ -antennerna. Till att börja med simulerades en enkel patch-antenn utan inset feed. Eftersom en sådan antenn inte går att matcha till 50Ω inimpedans valdes att därefter koncentrera arbetet på antenner med så kallad inset feed. Dessa antenner är relativt enkla att matcha till en önskad impedans.

Viss simulering av cirkulära patch-antennerna har även utförts.

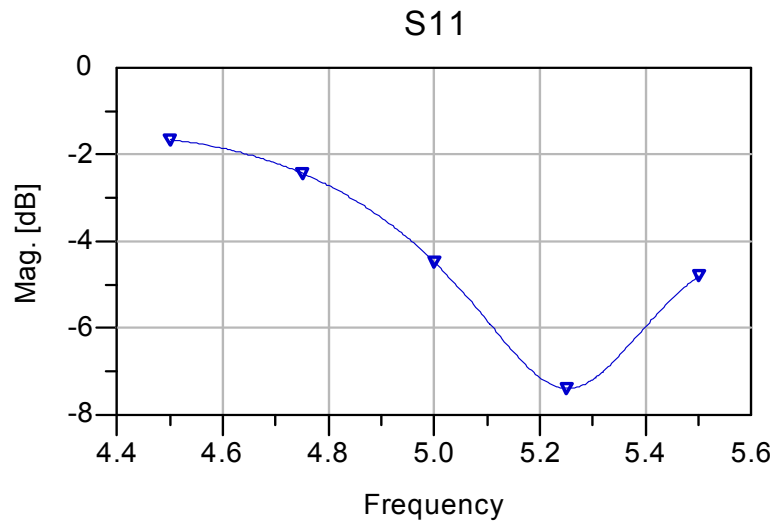
$\lambda/2$ -antenn utan inset feed, Layout



Figuren ovan visar layouten på en enkel patch-antenn utan inset feed.

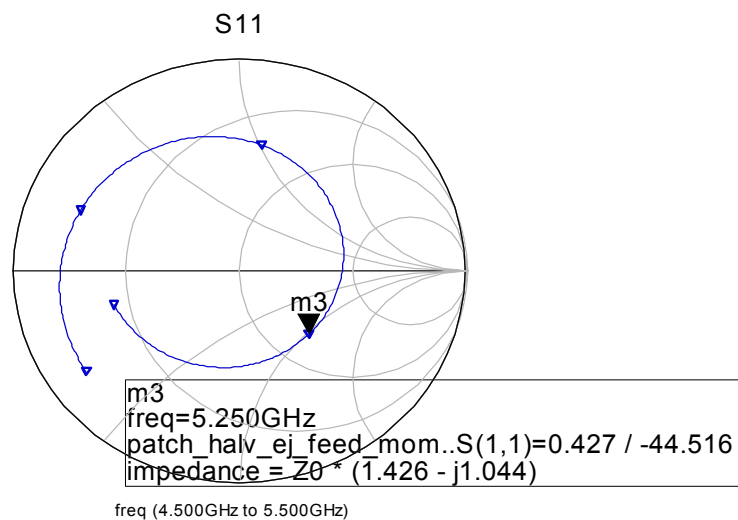
Antennen har följande mått (mm); Bredd patch=17.22, Längd patch=13.23, Längd Microstrip=31, Bredd Microstrip=1.88.

$\lambda/2$ -antenn utan inset feed, Reflektion



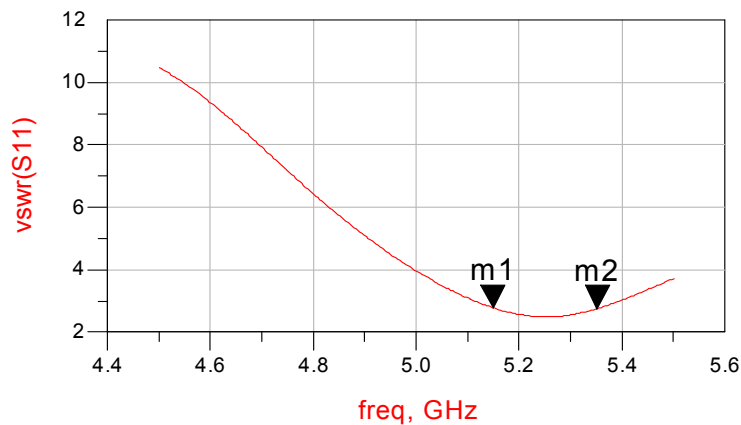
Figuren ovan visar att önskad resonansfrekvens har uppnåtts (5.25GHz), dock är magnituden för hög. Reflektionen skall vara så låg som möjligt för att möjliggöra att högsta möjliga effekt ut från antennen.

$\lambda/2$ -antenn utan inset feed, Matchning



Figuren visar att antennen inte är matchad till 50 Ω . Impedansen i punkten m3 är ca 88 Ω .

$\lambda/2$ -antenn utan inset feed, VSWR



m1
freq=5.148GHz
vswr(S11)=2.781

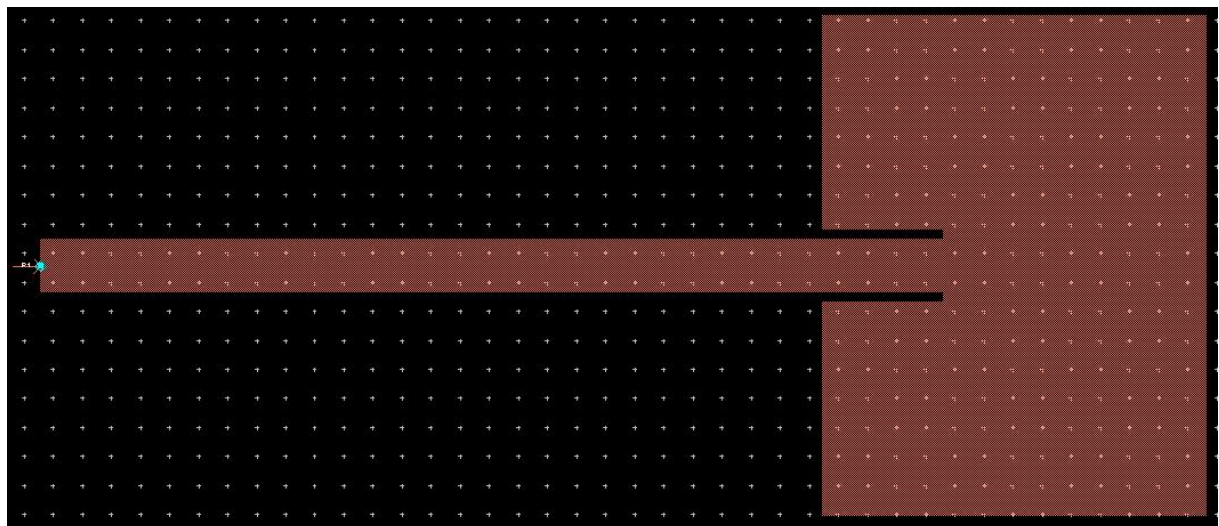
m2
freq=5.352GHz
vswr(S11)=2.752

Figuren visar att vid en bandbredd på 200 MHz (5.15-5.35GHz) erhålls en *Voltage Standing Wave Ratio* på ca 2.7.

Slutsats $\lambda/2$ -antenn utan inset feed

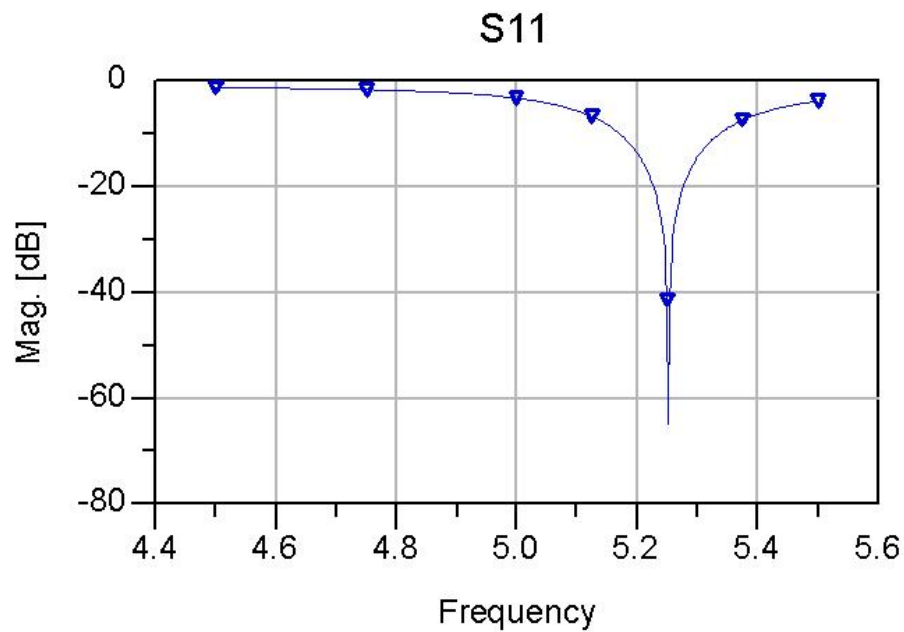
Som simuleringarna visar går det inte att matcha en patch-antenn utan att använda sig av inset feed. Dock verkar de beräknade måtten stämma bra, eftersom resonansfrekvensen blir rätt. VSWR är också ganska bra. Det som återstår är alltså att lösa problemet med matchningen.

$\lambda/2$ -antenn med inset feed, Layout



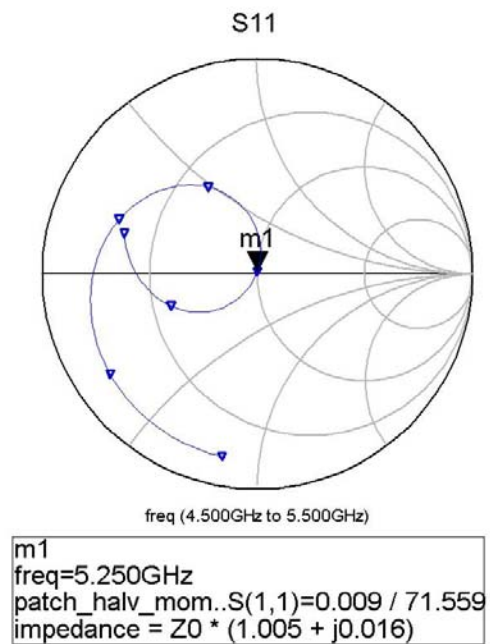
Figuren ovan visar layouten på en rektangulär patch-antenn med inset feed. Antennen har följande mått (mm); Bredd patch=17.22, Längd patch=13.23, Längd Microstrip=31, Bredd Microstrip=1.88, Inset feed $X_0=4.15$ $Y_0=0.3$

$\lambda/2$ -antenn med inset feed, Reflektion



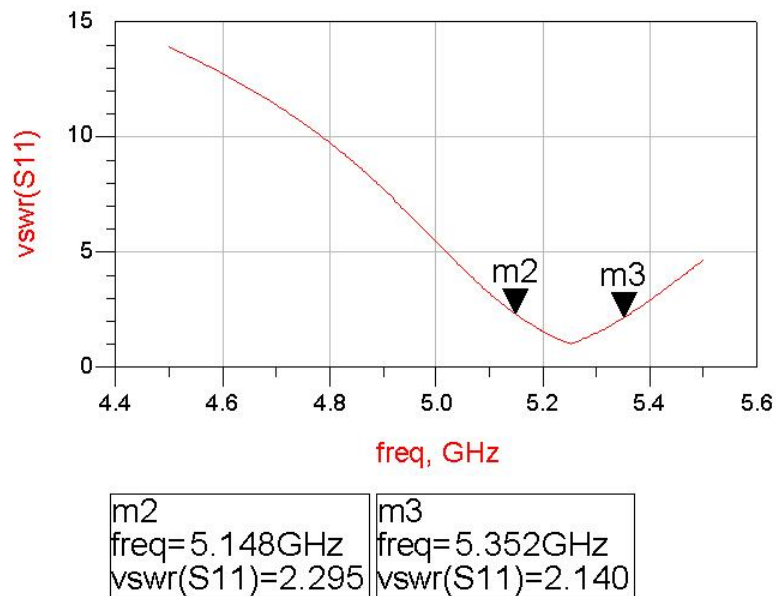
Figuren visar att antennen har mycket bra reflektion vid resonansfrekvensen.

$\lambda/2$ -antenn med inset feed, Matchning



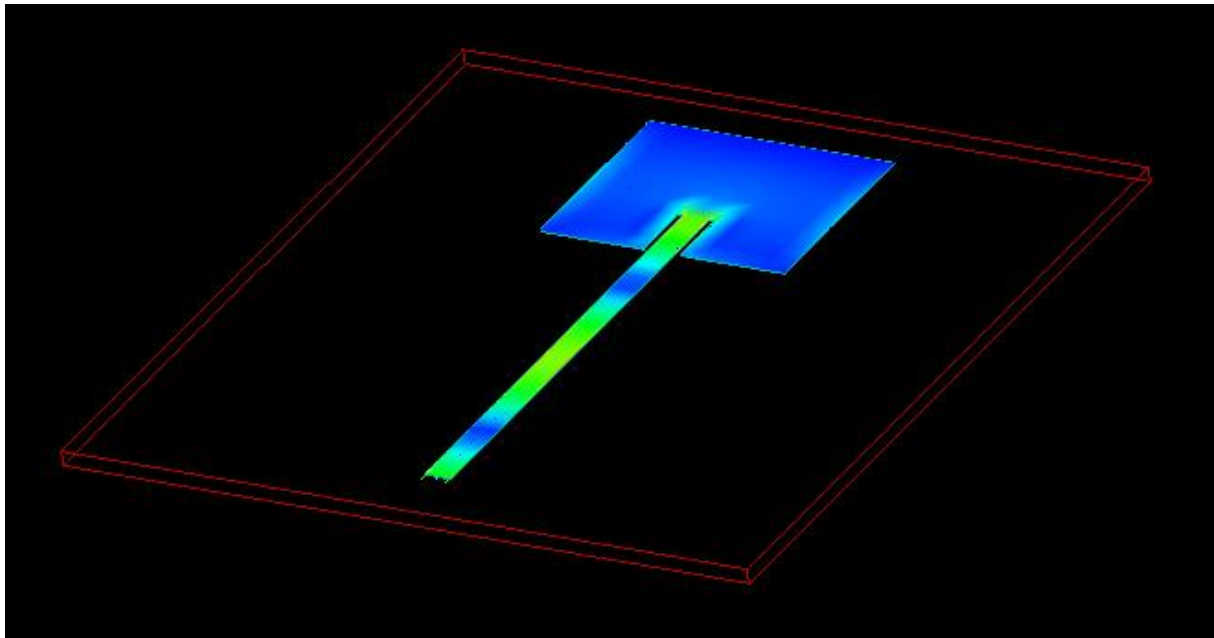
Mycket bra matchning vid 5.25GHz.

$\lambda/2$ -antenn med inset feed, VSWR



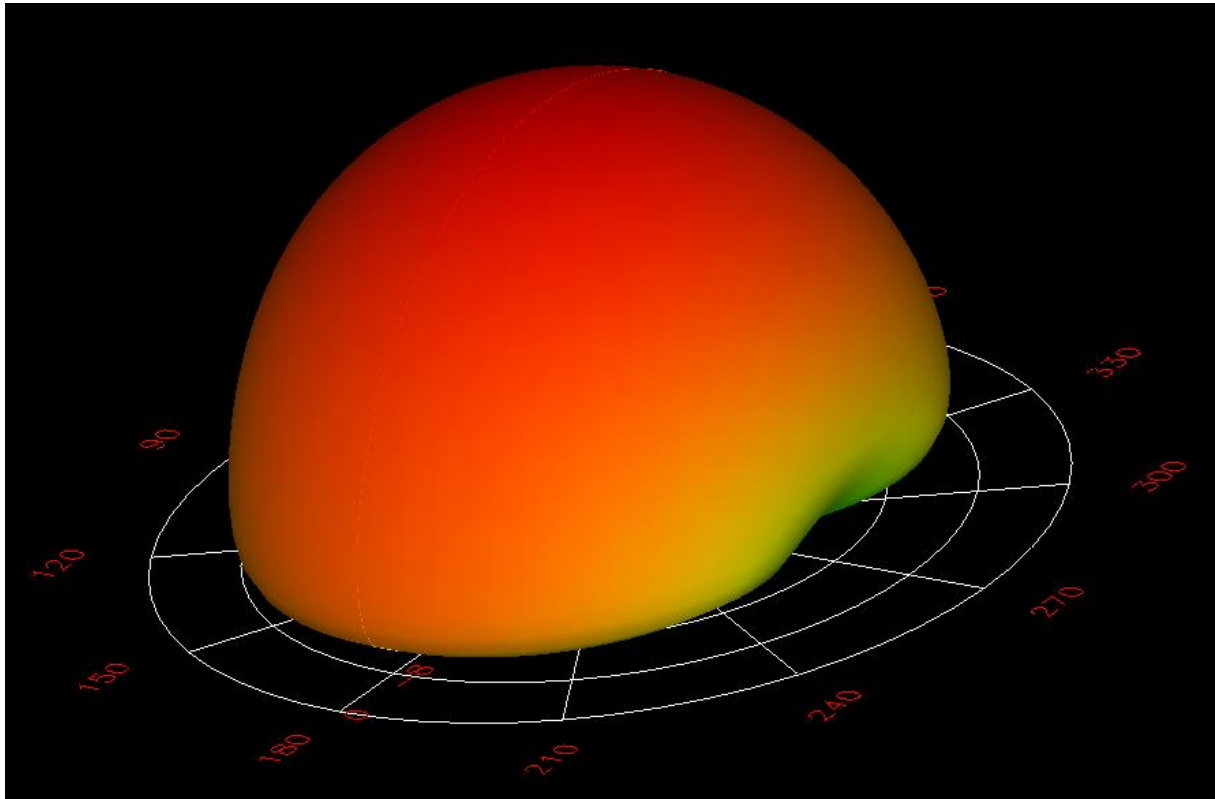
Som kan ses i figuren ovan har VSWR minskat till ca 2.3.

$\lambda/2$ -antenn med inset feed, Strömmar



Figuren visar hur strömmen flyter i antennen. För att få en bättre uppfattning om strömmens uppförande bör momentums animeringsfunktion användas. Detta är inget som gruppen har lagt ner stor vikt på vid simuleringarna.

$\lambda/2$ -antenn med inset feed, E-fält



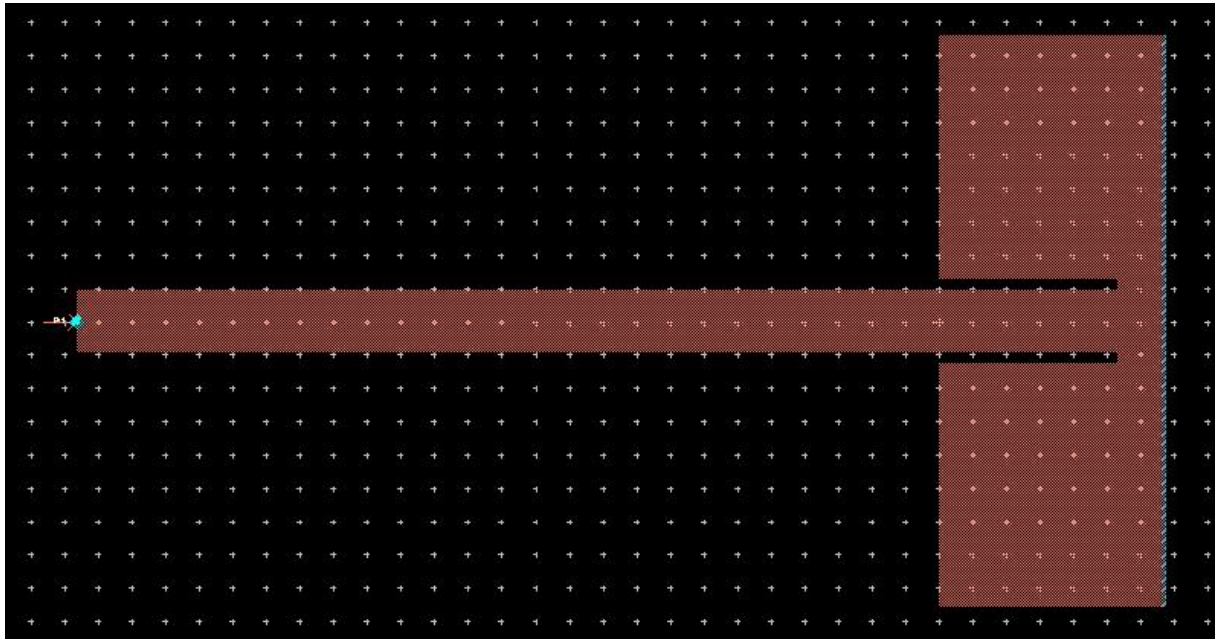
Figuren visar hur E-fältet strålar ut från antennen. Det som kan kommenteras är att eftersom det sitter ett jordplan under antennen, som är oändligt stort, strålar antennen endast uppåt och åt sidorna.

Slutsats $\lambda/2$ -antenn med inset feed

De slutsatser gruppen har dragit av simuleringarna är följande;

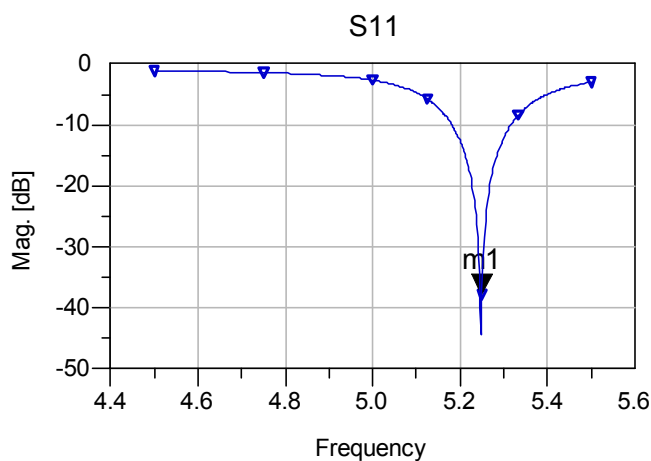
- o Med inset feed fås en mycket bra matchning
- o Reflektionen minskas, detta beror på matchningen
- o VSWR minskar till ca 2.3

$\lambda/4$ -antenn med inset feed, Layout



Eftersom impedansen i mitten av $\lambda/2$ -vågsantennen är 0Ω placeras en anslutning till jord där. Resultatet är fysiskt en antenn som är hälften så stor som $\lambda/2$ -vågsantennen. Antennen har följande mått (mm); Bredd patch=17.22, Längd patch=6.64, Längd Microstrip=31, Bredd Microstrip=1.88, Inset feed $X_0=5.33$ $Y_0=0.3$

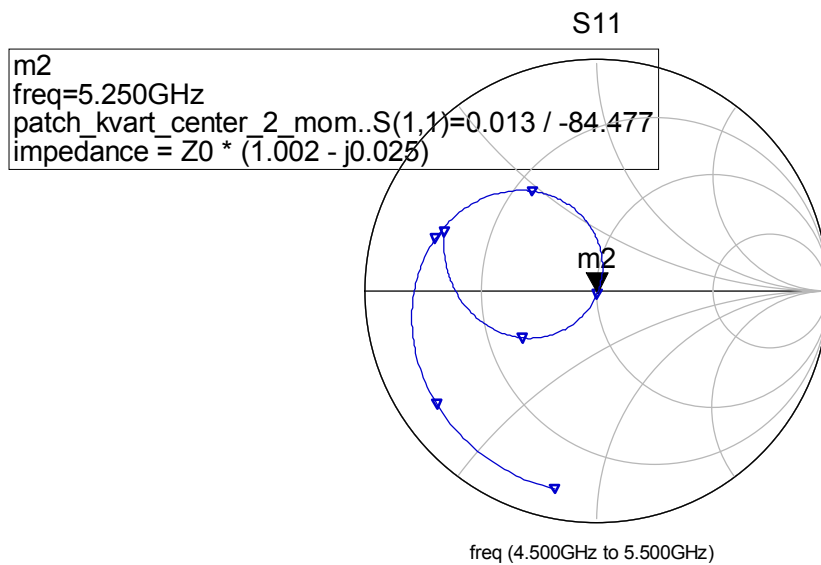
$\lambda/4$ -antenn med inset feed, Reflektion



m1 freq=5.250GHz dB(patch_kvart_center_2_mom..S(1,1))=-37.927

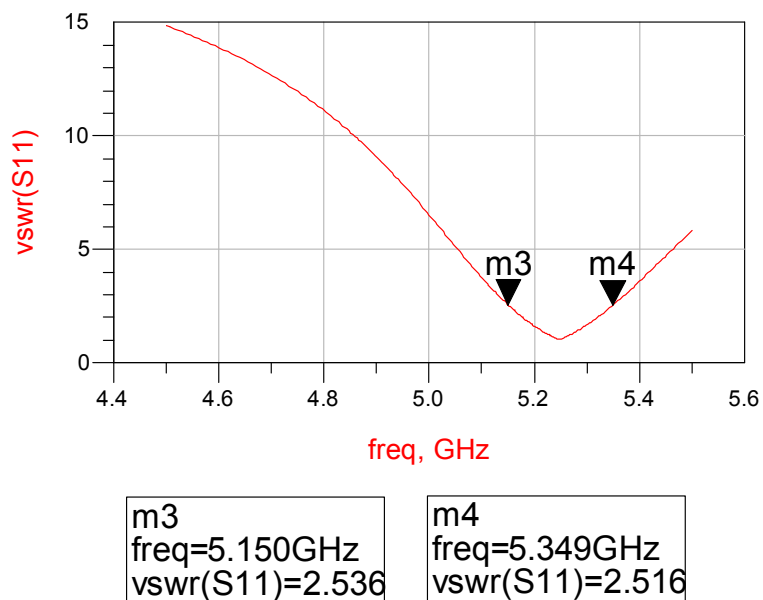
Reflektionen är likvärdig med en $\lambda/2$ -antenn.

$\lambda/4$ -antenn med inset feed, Matchning



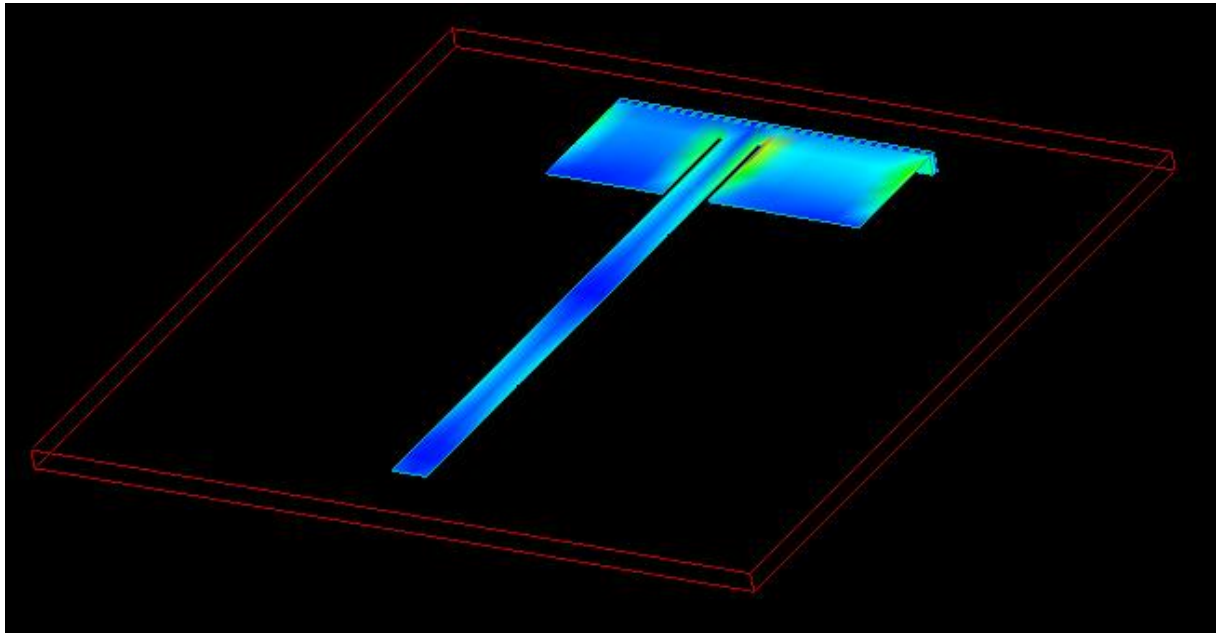
Antennen är matchad till 50Ω .

$\lambda/4$ -antenn med inset feed, VSWR

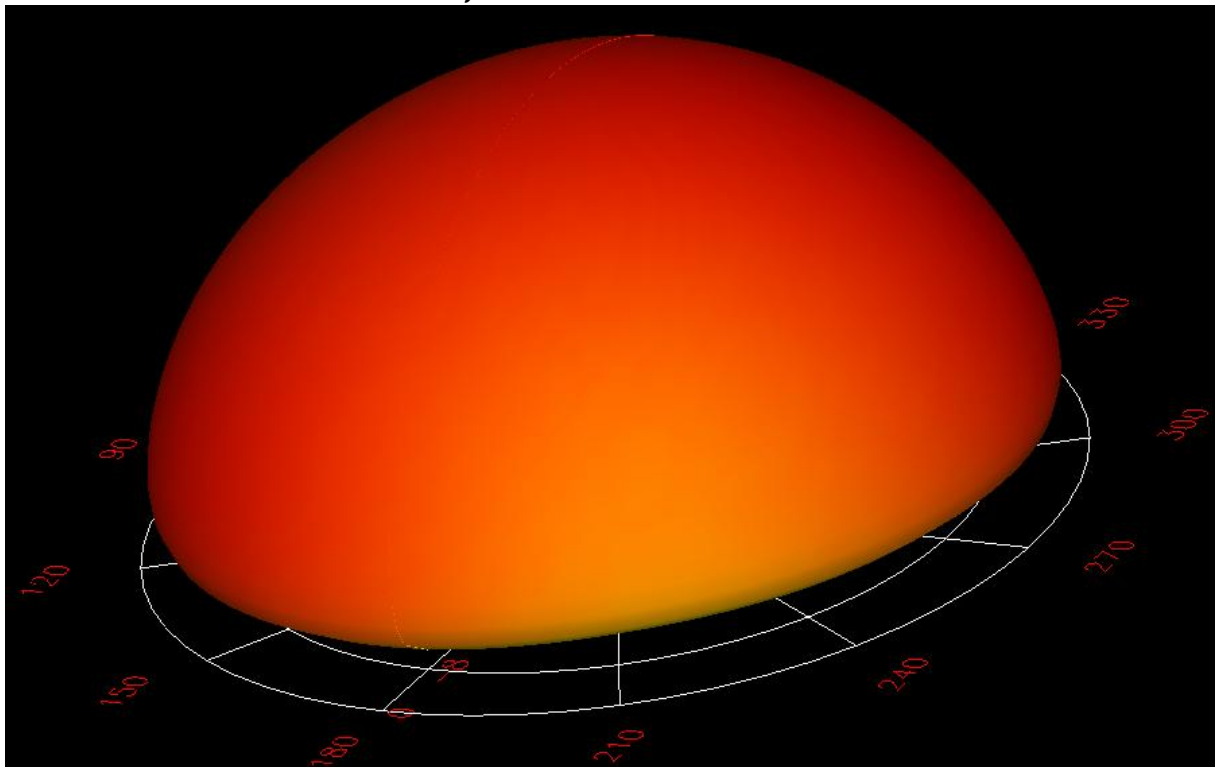


Som kan ses i figuren ovan är VSWR något sämre än för motsvarande $\lambda/2$ - antenn.

$\lambda/4$ -antenn med inset feed, Strömmar



$\lambda/4$ -antenn med inset feed, E-fält

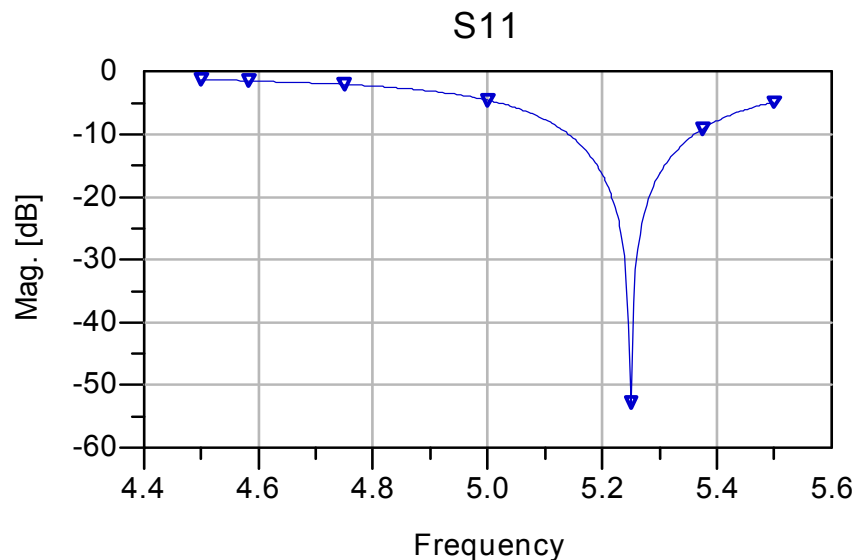


En något rundare spridning än för $\lambda/2$ -antennen.

Slutsats $\lambda/4$ -antenn med inset feed

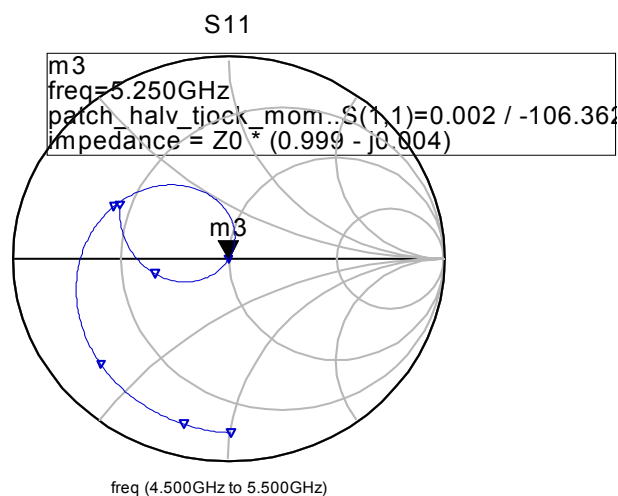
De slutsatser gruppen har dragit av simuleringarna är att storleken på antennen inte har så stor betydelse vad gäller reflektion eller matchning. Det som bör nämnas är att antennis förstärkning sjunker, detta beror på att antennis area är mindre.

$\lambda/2$ -antenn med inset feed, 1.4 mm substrat, Reflektion



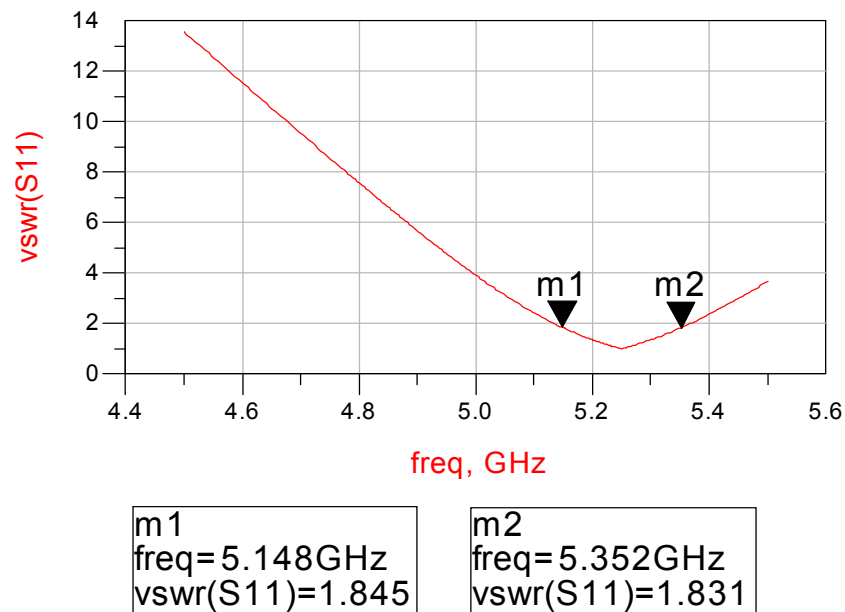
Vad användande av FR4-substrat med en tjocklek på 1.4 mm fås en mycket låg reflektion.

$\lambda/2$ -antenn med inset feed, 1.4 mm substrat, Matchning



Antennen är fortfarande matchad till 50Ω , vid frekvensen 5.25GHz.

$\lambda/2$ -antenn med inset feed, 1.4 mm substrat, VSWR



VSWR sjunker så lågt som 1.8, med en bandbredd på 200 MHz.

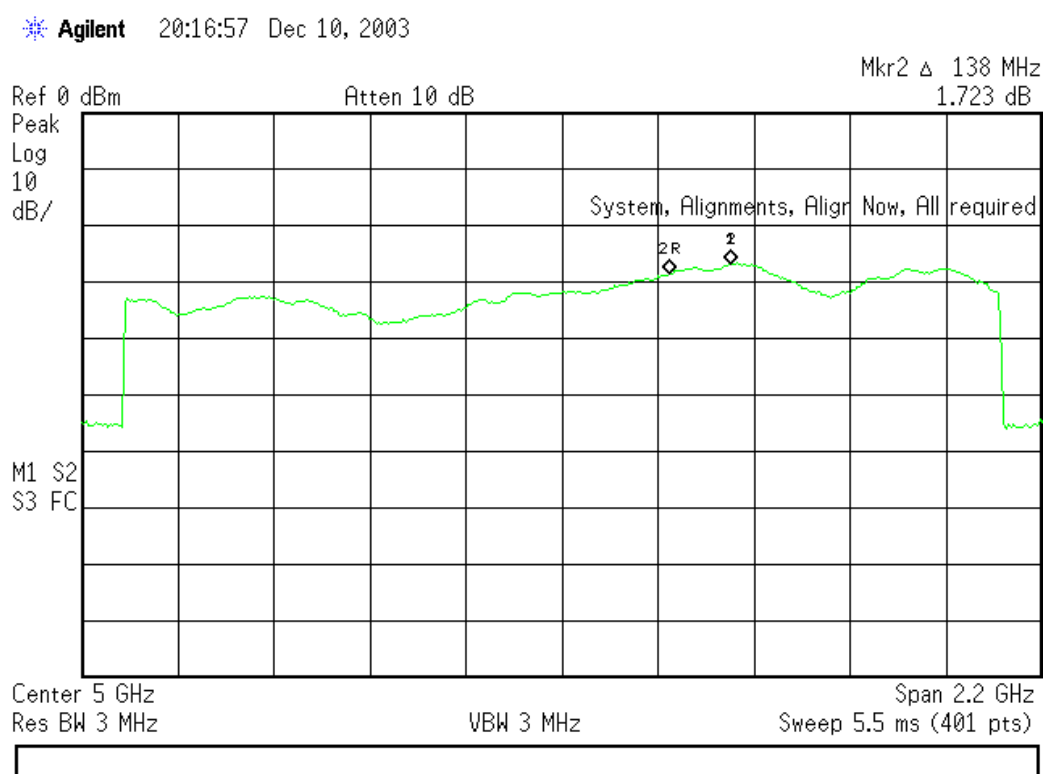
Resultat/Jämförelser simulering

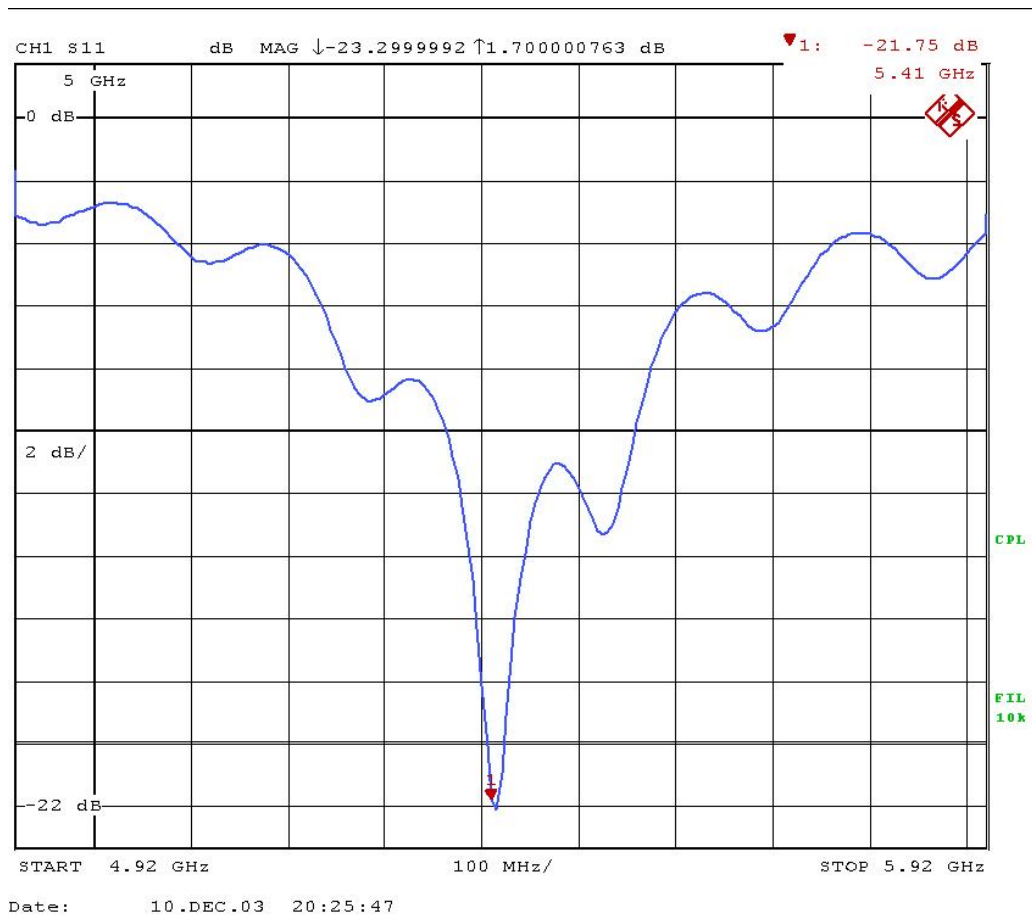
En $\lambda/2$ -antenn har en bättre förstärkning än en $\lambda/4$ -antenn, $\lambda/2$ har även en större bandbredd, det vill säga VSWR är lägre. Det som kan vara en fördel för $\lambda/4$ -antennerna är dess fysiska storlek. För att nå målet med VSWR=2 är det enklaste sättet att använda sig av ett tjockare substrat, detta gör att bandbredden ökar markant.

Verifiering

Gruppen har som ett sista led i projektet konstruerat ett 1 mm mönsterkort med en $\lambda/2$ -antenn på. Gerber-filer för konstruktion av antennerna på mönsterkort exporterades från ADS. Dessa filer användes sedan i mönsterkortslaboratoriet för att generera de filmer som behövs för processen.

För att verifiera antennens karaktäristik gjordes ett enkelt frekvenssvep med hjälp av signalgeneratoren SMIQ06B mellan 4 och 6GHz. Detta är samma frekvensintervall som har använts vid simuleringar i ADS. Signalen analyserades med hjälp av spektrumanalysatorn Agilent B4407. I figuren nedan visas den uppmätta signalen. Markören 2R visar frekvensen 5,25 GHz. Maximal resonans erhöles vid 5,39 GHz. Ett resultat som får anses bra.





I figuren ovan visas antennens reflektion. Antennen resonerar mest vid frekvens 5.4 GHz, ett resultat som gruppen är relativt nöjda med, med tanke på den stora skillnad i frekvens som en ändring på 1/10 mm gör.

Design av Patch-antenn

Vid design av en patch-antenn så finns det vissa parametrar som påverkar resultatet på ett väldigt tydligt sätt. I vissa fall så påverkar dock en parameter flera olika saker, samt att en parameter påverkar en annan. Detta gör att det behövs ganska så komplexa ekvationer för att kunna få en bild av hur antennen fungerar.

Det som är av störst betydelse är:

Val av substrat i patch-antenn:

Det finns en mängd olika substrat som skiljer sig väsentligt åt i egenskaper och pris. Substratets dielektriska konstant kommer framförallt att påverka bandbredden och hur effektivt antennen strålar. Där lägre permitivitet ger bredare impedansbandbredd och reducerad ytvågsexcitation

Substratets tjocklek:

Substratets tjocklek kommer att påverka bandbredden. Normalt så har man tjocklekar mellan 1 mm och 1,6 mm i FR4 substrat. Ju tjockare substratet är ju bättre bandbredd får antennen.

Patch-antennens längd:

Längden på patch-antennen bestämmer vid vilken frekvens som antennen sänder.

Patch-antennens bredd:

Bredden på patch-antennen påverkar resonansresistansen. En bredare patch ger lägre resistans. Helt fyrkantiga patch-antennar ger kross-polarisation och bör undvikas om man inte vill ha dubbel eller cirkulär polarisation.

Val av substrat i microstripledaren:

Ska väljas så att man får bra egenskaper på mikrokretsen, typiska värden på dielektricitets konstanten är mellan 2 och 10.

I vårt fall så har vi samma substrat i microstripen som i patch-antennen.

Microstripledarens tjocklek:

Tunnare microstrip-substrat ger mindre strålning från microstripen, dock ger det högre förluster. $0,01\lambda$ till $0,02\lambda$ brukar vara en bra kompromiss.

I vårt fall så har vi samma tjocklek i microstripen som i patch-antennen.

Microstripledarens bredd:

Ska matchas till 50 Ohm.

Resultat och slutsats

Projektet har resulterat i ökad förståelse för hur en patch-antenn fungerar och är uppbyggd. Då främst design av enlagers antenner. Gruppen har fått såväl teoretiska som praktiska kunskaper i hur en patch antenn bör designas för att den inte bara ska sända vid önskad frekvens utan även ha så låg SWR som möjligt samt högsta möjliga bandbredd.

För att nå våra mål så har gruppen konstruerat ett eget Matlab program.

I detta program så kan man räkna ut längd, bredd samt inset feed point.

För halvvågs och kvartsvågs antenner.

Samt radien för cirkulära.

De resultat som man får ut behöver sen bara fin justeras för att få en mycket bra antenn.

För $\lambda/2$ antennen så var det första målet att ha en bandbredd på 200 MHz vid 5,25 GHz som hade en SWR under 3.

Detta visade sig inte vara några som helst problem. Så vi fokuserade på att få en så stor bandbredd som möjligt vid SWR 2. Här fick vi fram en maximalbandbredd på 170 MHz, samt en gain på 3,7 dB

För $\lambda/4$ antennen så fick vi från början en SWR på 2,5. Alltså bättre än SWR ratio 3.

Detta gjorde att vi även koncentrerade oss på att få så bred bandbredd som möjligt vid SWR 2. Här fick vi fram en maximal bandbredd på 150 MHz, samt en gain på 0,4 dB.

Gainen bör bli lägre för $\lambda/4$, pga av arean är mindre. Vi tycker dock att det skiljer lite väl mycket.

Vi testade även att simulera cirkulära antenner. De visade sig vara sämre på det mesta och vi lade ingen tyngd på dem, eftersom att det inte ingick i projektet.

Slutligen testades olika tjocklekar av substrat och då förbättrades resultatet kraftigt.

För att verifiera simuleringarna så tillverkades en rektangulär patch-antenn. Denna uppmättes sedan med hjälp av en vektoranalysator med lyckat resultat

Referenser

- [1] Garg, Ramesh (2001), "*Microstrip antenna design handbook*", Norwood, NA: Artech House, Inc. ISBN 0-89006-513-6.
- [2] Zürcher, Jean-François (1995), "*Broadband patch antennas*", Norwood, NA: Artech House, Inc. ISBN 0-89006-777-5.
- [3] Kildal, Per-Simon (2000), "*Foundations of antennas, a unified approach*", Lund: Studentlitteratur. ISBN 91-44-01322-1.
- [4] Balanis, C.A. "*Antenna Theory Analysis and Design, Second edition*", United States of America: John Wiley & Sons 1997.
- [5] Schneider, M. V. (Jan. 1972) "*Microstrip dispersion*" Proc. IEEE, vol. 60, no 1, pp. 144-146.
- [6] Bhartia P., et al. (1991) "*Millimeter-Wave Microstrip and Printed Circuit Antennas*", p10. Norwood, Mass: Artech House.
- [7] James J.R, P.S. Hall and C. Wood. (1981) "*Microstrip Antenna Theory and Design*", pp 87-89. London, United Kingdom. Peter Peregrinus.
- [8] Harrington, R. F. (1996) "*Time-Harmonic Electromagnetic Waves*", New York: McGraw-Hill.
- [9] A Review of Aperture Coupled Microstrip Antennas: History, Operation, Development, and Applications
<http://www.ecs.umass.edu/ece/pozar/aperture.pdf> (2003-12-10)
- [10] Reinhold, Ludwig and Pavel, Bretchko. (2000) "*RF Circuit Design*", Upper saddle river , New Jersey, Prentice and Hall. ISBN 0-13-095323-7
- [11] Pozar, David M and Schaubert, Daniel H. (1995) "*Microstrip Antennas*", IEEE Press

Bilaga 1 – Matlab kod

```
function varargout = MicroStrip_PatchV3_1(varargin)
% MICROSTRIP_PATCHV3_1 M-file for MicroStrip_PatchV3_1.fig
%     MICROSTRIP_PATCHV3_1, by itself, creates a new MICROSTRIP_PATCHV3_1
or raises the existing
%     singleton*.
%
%     H = MICROSTRIP_PATCHV3_1 returns the handle to a new
MICROSTRIP_PATCHV3_1 or the handle to
%     the existing singleton*.
%
%     MICROSTRIP_PATCHV3_1('CALLBACK',hObject,eventData,handles,...) calls
the local
%     function named CALLBACK in MICROSTRIP_PATCHV3_1.M with the given
input arguments.
%
%     MICROSTRIP_PATCHV3_1('Property','Value',...) creates a new
MICROSTRIP_PATCHV3_1 or raises the
%     existing singleton*. Starting from the left, property value pairs
are
%     applied to the GUI before MicroStrip_PatchV3_1_OpeningFunction gets
called. An
%     unrecognized property name or invalid value makes property
application
%     stop. All inputs are passed to MicroStrip_PatchV3_1_OpeningFcn via
varargin.
%
%     *See GUI Options on GUIDE's Tools menu. Choose "GUI allows only one
%     instance to run (singleton)".
%
% See also: GUIDE, GUIDATA, GUIHANDLES

% Edit the above text to modify the response to help MicroStrip_PatchV3_1

% Last Modified by GUIDE v2.5 02-Dec-2003 19:40:16

% Begin initialization code - DO NOT EDIT
gui_Singleton = 1;
gui_State = struct('gui_Name',       mfilename, ...
                  'gui_Singleton',   gui_Singleton, ...
                  'gui_OpeningFcn', @MicroStrip_PatchV3_1_OpeningFcn, ...
                  'gui_OutputFcn',  @MicroStrip_PatchV3_1_OutputFcn, ...
                  'gui_LayoutFcn',  [], ...
                  'gui_Callback',    []);
if nargin & isstr(varargin{1})
    gui_State.gui_Callback = str2func(varargin{1});
end

if nargout
    [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
    gui_mainfcn(gui_State, varargin{:});
end
% End initialization code - DO NOT EDIT

% --- Executes just before MicroStrip_PatchV3_1 is made visible.
function MicroStrip_PatchV3_1_OpeningFcn(hObject, eventdata, handles,
varargin)
```

```

% This function has no output args, see OutputFcn.
% hObject    handle to figure
% eventdata  reserved - to be defined in a future version of MATLAB
% handles     structure with handles and user data (see GUIDATA)
% varargin    command line arguments to MicroStrip_PatchV3_1 (see VARARGIN)

% Choose default command line output for MicroStrip_PatchV3_1
handles.output = hObject;

% Update handles structure
guidata(hObject, handles);

% UIWAIT makes MicroStrip_PatchV3_1 wait for user response (see UIRESUME)
% uiwait(handles.figure1);

% --- Outputs from this function are returned to the command line.
function varargout = MicroStrip_PatchV3_1_OutputFcn(hObject, eventdata,
handles)
% varargout    cell array for returning output args (see VARARGOUT);
% hObject      handle to figure
% eventdata    reserved - to be defined in a future version of MATLAB
% handles       structure with handles and user data (see GUIDATA)

%Initiering av GUI
%Rectangular Patch Antenna Half Wavelength
set(handles.Rec2HSubstrate, 'String', num2str(1.0))
set(handles.Rec2Permattivity, 'String', num2str(4.5+0.02i))
set(handles.Rec2PatchFreq, 'String', num2str(5.25e9))

%Rectangular Patch Antenna Quarter Wavelength
set(handles.Rec4HSubstrate, 'String', num2str(1.0))
set(handles.Rec4Permattivity, 'String', num2str(4.5+0.02i))
set(handles.Rec4PatchFreq, 'String', num2str(5.25e9))

%Circular Patch Antenna
set(handles.CirHSubstrate, 'String', num2str(1.0))
set(handles.CirPermattivity, 'String', num2str(4.5+0.02i))
set(handles.CirPatchFreq, 'String', num2str(5.25e9))

% Get default command line output from handles structure
varargout{1} = handles.output;

% --- Executes on button press in Calc.
function Calc_Callback(hObject, eventdata, handles)
% hObject      handle to Calc (see GCBO)
% eventdata    reserved - to be defined in a future version of MATLAB
% handles       structure with handles and user data (see GUIDATA)

%Uträkning av rektangulär Patch Lamda/2
%hradio = [handles.radiobutton2,handles.radiobutton3,handles.radiobutton4];

Rec2h = str2num(get(handles.Rec2HSubstrate, 'String'));
Rec2Er = str2num(get(handles.Rec2Permattivity, 'String'));
Rec2Fresgoal = str2num(get(handles.Rec2PatchFreq, 'String'));

A = Rectangular2_patch_antenna_V3_1(Rec2h*0.001, Rec2Er, Rec2Fresgoal);

set(handles.Rec2LPatch, 'String', num2str(A(8)*1000))
set(handles.Rec2WPatch, 'String', num2str(A(20)))

```

```

set(handles.Rec2PatchFreqGoal, 'String', num2str(A(9)))
set(handles.Rec2PatchEdgeImp, 'String', num2str(A(16)))
set(handles.Rec2FeedPointImp, 'String', num2str(A(18)))
set(handles.Rec2FeedPoint, 'String', num2str(A(17)*1000))
set(handles.Rec2LFeedLine, 'String', num2str(A(12)*1000))
set(handles.Rec2WFeedLine, 'String', num2str(A(11)*1000))
set(handles.Rec2ImpFeedLine, 'String', num2str(A(14)))

Rec4h = str2num(get(handles.Rec4HSubstrate, 'String'));
Rec4Er = str2num(get(handles.Rec4Permittivity, 'String'));
Rec4Fresgoal = str2num(get(handles.Rec4PatchFreq, 'String'));

B = Rectangular4_patch_antenna_V3_1(Rec4h*0.001, Rec4Er, Rec4Fresgoal);

set(handles.Rec4LPatch, 'String', num2str(B(8)*1000))
set(handles.Rec4WPatch, 'String', num2str(B(20)))
set(handles.Rec4PatchFreqGoal, 'String', num2str(B(9)))
set(handles.Rec4PatchEdgeImp, 'String', num2str(B(16)))
set(handles.Rec4FeedPointImp, 'String', num2str(B(18)))
set(handles.Rec4FeedPoint, 'String', num2str(B(17)*1000))
set(handles.Rec4LFeedLine, 'String', num2str(B(12)*1000))
set(handles.Rec4WFeedLine, 'String', num2str(B(11)*1000))
set(handles.Rec4ImpFeedLine, 'String', num2str(B(14)))

Cirh = str2num(get(handles.CirHSubstrate, 'String'));
CirEr = str2num(get(handles.CirPermittivity, 'String'));
CirFresgoal = str2num(get(handles.CirPatchFreq, 'String'));

C = Circular_patch_antenna_V3_1(Cirh*0.001, CirEr, CirFresgoal);

set(handles.CirPatchRadie, 'String', num2str(C(1)*1000))
set(handles.CirPatchFreqGoal, 'String', num2str(C(6)))
%%%%%%
set(handles.CirPatchEdgeImp, 'String', num2str(0))
set(handles.CirFeedPointImp, 'String', num2str(0))
set(handles.CirFeedPoint, 'String', num2str(0))
%%%%%%
set(handles.CirLFeedLine, 'String', num2str(C(3)*1000))
set(handles.CirWFeedLine, 'String', num2str(C(4)*1000))
set(handles.CirImpFeedLine, 'String', num2str(C(5)))

% --- Executes on button press in Reset.
function Reset_Callback(hObject, eventdata, handles)
% hObject      handle to Reset (see GCBO)
% eventdata    reserved - to be defined in a future version of MATLAB
% handles      structure with handles and user data (see GUIDATA)

set(handles.Rec2LPatch, 'String', num2str(0))
set(handles.Rec2WPatch, 'String', num2str(0))
set(handles.Rec2PatchFreqGoal, 'String', num2str(0))
set(handles.Rec2PatchEdgeImp, 'String', num2str(0))
set(handles.Rec2FeedPointImp, 'String', num2str(0))
set(handles.Rec2FeedPoint, 'String', num2str(0))
set(handles.Rec2LFeedLine, 'String', num2str(0))
set(handles.Rec2WFeedLine, 'String', num2str(0))
set(handles.Rec2ImpFeedLine, 'String', num2str(0))

set(handles.Rec4LPatch, 'String', num2str(0))
set(handles.Rec4WPatch, 'String', num2str(0))
set(handles.Rec4PatchFreqGoal, 'String', num2str(0))
set(handles.Rec4PatchEdgeImp, 'String', num2str(0))

```

```

set(handles.Rec4FeedPointImp,'String',num2str(0))
set(handles.Rec4FeedPoint,'String',num2str(0))
set(handles.Rec4LFeedLine,'String',num2str(0))
set(handles.Rec4WFeedLine,'String',num2str(0))
set(handles.Rec4ImpFeedLine,'String',num2str(0))

set(handles.CirPatchRadie,'String',num2str(0))
set(handles.CirPatchFreqGoal,'String',num2str(0))
set(handles.CirPatchEdgeImp,'String',num2str(0))
set(handles.CirFeedPointImp,'String',num2str(0))
set(handles.CirFeedPoint,'String',num2str(0))
set(handles.CirLFeedLine,'String',num2str(0))
set(handles.CirWFeedLine,'String',num2str(0))
set(handles.CirImpFeedLine,'String',num2str(0))

% --- Executes on button press in Exit.
function Exit_Callback(hObject, eventdata, handles)
% hObject      handle to Exit (see GCBO)
% eventdata    reserved - to be defined in a future version of MATLAB
% handles      structure with handles and user data (see GUIDATA)

% --- Executes during object creation, after setting all properties.
function Rec2HSubstrate_CreateFcn(hObject, eventdata, handles)
% hObject      handle to Rec2HSubstrate (see GCBO)
% eventdata    reserved - to be defined in a future version of MATLAB
% handles      empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function Rec2HSubstrate_Callback(hObject, eventdata, handles)
% hObject      handle to Rec2HSubstrate (see GCBO)
% eventdata    reserved - to be defined in a future version of MATLAB
% handles      structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2HSubstrate as text
%         str2double(get(hObject,'String')) returns contents of
Rec2HSubstrate as a double

% --- Executes during object creation, after setting all properties.
function edit2_CreateFcn(hObject, eventdata, handles)
% hObject      handle to edit2 (see GCBO)
% eventdata    reserved - to be defined in a future version of MATLAB
% handles      empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

```

```
set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function edit2_Callback(hObject, eventdata, handles)
% hObject    handle to edit2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of edit2 as text
%        str2double(get(hObject, 'String')) returns contents of edit2 as a
double
```

```
% --- Executes during object creation, after setting all properties.
function Rec2PatchFreq_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2PatchFreq (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%        See ISPC and COMPUTER.
if ispc
    set(hObject, 'BackgroundColor', 'white');
else

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function Rec2PatchFreq_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2PatchFreq (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of Rec2PatchFreq as text
%        str2double(get(hObject, 'String')) returns contents of
Rec2PatchFreq as a double
```

```
% --- Executes during object creation, after setting all properties.
function Rec2Permittivity_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2Permittivity (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%        See ISPC and COMPUTER.
if ispc
    set(hObject, 'BackgroundColor', 'white');
else

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function Rec2Permittivity_Callback(hObject, eventdata, handles)
```

```

% hObject    handle to Rec2Permittivity (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2Permittivity as text
%         str2double(get(hObject,'String')) returns contents of
Rec2Permittivity as a double

% --- Executes during object creation, after setting all properties.
function Rec2LPatch_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2LPatch (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function Rec2LPatch_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2LPatch (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2LPatch as text
%         str2double(get(hObject,'String')) returns contents of Rec2LPatch
as a double

% --- Executes during object creation, after setting all properties.
function edit9_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit9 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function edit9_Callback(hObject, eventdata, handles)
% hObject    handle to edit9 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit9 as text
%         str2double(get(hObject,'String')) returns contents of edit9 as a
double

```

```

% --- Executes during object creation, after setting all properties.
function Rec2WPatch_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2WPatch (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function Rec2WPatch_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2WPatch (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2WPatch as text
%         str2double(get(hObject,'String')) returns contents of Rec2WPatch
%         as a double


% --- Executes during object creation, after setting all properties.
function Rec2PatchEdgeImp_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2PatchEdgeImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function Rec2PatchEdgeImp_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2PatchEdgeImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2PatchEdgeImp as text
%         str2double(get(hObject,'String')) returns contents of
%         Rec2PatchEdgeImp as a double


% --- Executes during object creation, after setting all properties.
function Rec2FeedPointImp_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2FeedPointImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

```



```

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function Rec2FeedPointImp_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2FeedPointImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2FeedPointImp as text
%        str2double(get(hObject,'String')) returns contents of
Rec2FeedPointImp as a double


% --- Executes during object creation, after setting all properties.
function Rec2FeedPoint_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2FeedPoint (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function Rec2FeedPoint_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2FeedPoint (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2FeedPoint as text
%        str2double(get(hObject,'String')) returns contents of
Rec2FeedPoint as a double


% --- Executes during object creation, after setting all properties.
function Rec2LFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2LFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

```

```

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end

```

```

function Rec2LFeedLine_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2LFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of Rec2LFeedLine as text
%        str2double(get(hObject, 'String')) returns contents of
Rec2LFeedLine as a double

```

```

% --- Executes during object creation, after setting all properties.
function Rec2ImpFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2ImpFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%        See ISPC and COMPUTER.
if ispc
    set(hObject, 'BackgroundColor', 'white');
else

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end

```

```

function Rec2ImpFeedLine_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2ImpFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of Rec2ImpFeedLine as text
%        str2double(get(hObject, 'String')) returns contents of
Rec2ImpFeedLine as a double

```

```

% --- Executes during object creation, after setting all properties.
function Rec2WFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2WFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%        See ISPC and COMPUTER.
if ispc
    set(hObject, 'BackgroundColor', 'white');
else

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end

```

```

function Rec2WFeedLine_Callback(hObject, eventdata, handles)

```

```

% hObject    handle to Rec2WFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2WFeedLine as text
%         str2double(get(hObject,'String')) returns contents of
Rec2WFeedLine as a double

% --- Executes during object creation, after setting all properties.
function Rec4HSubstrate_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4HSubstrate (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function Rec4HSubstrate_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4HSubstrate (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4HSubstrate as text
%         str2double(get(hObject,'String')) returns contents of
Rec4HSubstrate as a double

% --- Executes during object creation, after setting all properties.
function Rec4PatchFreq_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4PatchFreq (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function Rec4PatchFreq_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4PatchFreq (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4PatchFreq as text
%         str2double(get(hObject,'String')) returns contents of
Rec4PatchFreq as a double

```

```

% --- Executes during object creation, after setting all properties.
function Rec4Permittivity_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4Permittivity (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function Rec4Permittivity_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4Permittivity (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4Permittivity as text
%        str2double(get(hObject,'String')) returns contents of
Rec4Permittivity as a double


% --- Executes during object creation, after setting all properties.
function Rec4LPatch_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4LPatch (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function Rec4LPatch_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4LPatch (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4LPatch as text
%        str2double(get(hObject,'String')) returns contents of Rec4LPatch
as a double


% --- Executes during object creation, after setting all properties.
function Rec4PatchFreqGoal_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4PatchFreqGoal (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

```

```

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function Rec4PatchFreqGoal_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4PatchFreqGoal (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4PatchFreqGoal as
text
%          str2double(get(hObject,'String')) returns contents of
Rec4PatchFreqGoal as a double


% --- Executes during object creation, after setting all properties.
function Rec4WPatch_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4WPatch (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function Rec4WPatch_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4WPatch (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4WPatch as text
%          str2double(get(hObject,'String')) returns contents of Rec4WPatch
as a double


% --- Executes during object creation, after setting all properties.
function Rec4PatchEdgeImp_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4PatchEdgeImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

```

```
set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function Rec4PatchEdgeImp_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4PatchEdgeImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of Rec4PatchEdgeImp as text
%        str2double(get(hObject, 'String')) returns contents of
Rec4PatchEdgeImp as a double
```

```
% --- Executes during object creation, after setting all properties.
function Rec4FeedPointImp_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4FeedPointImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called
```

```
% Hint: edit controls usually have a white background on Windows.
%        See ISPC and COMPUTER.
```

```
if ispc
    set(hObject, 'BackgroundColor', 'white');
else
```

```
set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function Rec4FeedPointImp_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4FeedPointImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of Rec4FeedPointImp as text
%        str2double(get(hObject, 'String')) returns contents of
Rec4FeedPointImp as a double
```

```
% --- Executes during object creation, after setting all properties.
function Rec4FeedPoint_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4FeedPoint (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called
```

```
% Hint: edit controls usually have a white background on Windows.
%        See ISPC and COMPUTER.
```

```
if ispc
    set(hObject, 'BackgroundColor', 'white');
else
```

```
set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function Rec4FeedPoint_Callback(hObject, eventdata, handles)
```

```

% hObject    handle to Rec4FeedPoint (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4FeedPoint as text
%         str2double(get(hObject,'String')) returns contents of
Rec4FeedPoint as a double

% --- Executes during object creation, after setting all properties.
function Rec4LFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4LFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function Rec4LFeedLine_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4LFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4LFeedLine as text
%         str2double(get(hObject,'String')) returns contents of
Rec4LFeedLine as a double

% --- Executes during object creation, after setting all properties.
function Rec4ImpFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4ImpFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function Rec4ImpFeedLine_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4ImpFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec4ImpFeedLine as text
%         str2double(get(hObject,'String')) returns contents of
Rec4ImpFeedLine as a double

```

```

% --- Executes during object creation, after setting all properties.
function Rec4WFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec4WFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject, 'BackgroundColor', 'white');
else

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end


function Rec4WFeedLine_Callback(hObject, eventdata, handles)
% hObject    handle to Rec4WFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of Rec4WFeedLine as text
%         str2double(get(hObject, 'String')) returns contents of
Rec4WFeedLine as a double


% --- Executes during object creation, after setting all properties.
function CirHSubstrate_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirHSubstrate (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject, 'BackgroundColor', 'white');
else

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end


function CirHSubstrate_Callback(hObject, eventdata, handles)
% hObject    handle to CirHSubstrate (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of CirHSubstrate as text
%         str2double(get(hObject, 'String')) returns contents of
CirHSubstrate as a double


% --- Executes during object creation, after setting all properties.
function CirPatchFreq_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirPatchFreq (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

```



```

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function CirPatchFreq_Callback(hObject, eventdata, handles)
% hObject    handle to CirPatchFreq (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of CirPatchFreq as text
%        str2double(get(hObject,'String')) returns contents of CirPatchFreq
%        as a double


% --- Executes during object creation, after setting all properties.
function edit52_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit52 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function edit52_Callback(hObject, eventdata, handles)
% hObject    handle to edit52 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit52 as text
%        str2double(get(hObject,'String')) returns contents of edit52 as a
%        double


% --- Executes during object creation, after setting all properties.
function CirPatchRadie_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirPatchRadie (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

```

```
set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function CirPatchRadie_Callback(hObject, eventdata, handles)
% hObject    handle to CirPatchRadie (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of CirPatchRadie as text
%        str2double(get(hObject, 'String')) returns contents of
%        CirPatchRadie as a double
```

```
% --- Executes during object creation, after setting all properties.
function CirPatchFreqGoal_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirPatchFreqGoal (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%        See ISPC and COMPUTER.
if ispc
    set(hObject, 'BackgroundColor', 'white');
else

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function CirPatchFreqGoal_Callback(hObject, eventdata, handles)
% hObject    handle to CirPatchFreqGoal (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject, 'String') returns contents of CirPatchFreqGoal as text
%        str2double(get(hObject, 'String')) returns contents of
%        CirPatchFreqGoal as a double
```

```
% --- Executes during object creation, after setting all properties.
function edit55_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit55 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%        See ISPC and COMPUTER.
if ispc
    set(hObject, 'BackgroundColor', 'white');
else

set(hObject, 'BackgroundColor', get(0, 'defaultUicontrolBackgroundColor'));
end
```

```
function edit55_Callback(hObject, eventdata, handles)
```

```

% hObject    handle to edit55 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit55 as text
%         str2double(get(hObject,'String')) returns contents of edit55 as a
double

% --- Executes during object creation, after setting all properties.
function CirPatchEdgeImp_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirPatchEdgeImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function CirPatchEdgeImp_Callback(hObject, eventdata, handles)
% hObject    handle to CirPatchEdgeImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of CirPatchEdgeImp as text
%         str2double(get(hObject,'String')) returns contents of
CirPatchEdgeImp as a double

% --- Executes during object creation, after setting all properties.
function CirFeedPointImp_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirFeedPointImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end

function CirFeedPointImp_Callback(hObject, eventdata, handles)
% hObject    handle to CirFeedPointImp (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of CirFeedPointImp as text
%         str2double(get(hObject,'String')) returns contents of
CirFeedPointImp as a double

```

```

% --- Executes during object creation, after setting all properties.
function edit58_CreateFcn(hObject, eventdata, handles)
% hObject    handle to edit58 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function edit58_Callback(hObject, eventdata, handles)
% hObject    handle to edit58 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of edit58 as text
%         str2double(get(hObject,'String')) returns contents of edit58 as a
double


% --- Executes during object creation, after setting all properties.
function CirLFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirLFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%         See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function CirLFeedLine_Callback(hObject, eventdata, handles)
% hObject    handle to CirLFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of CirLFeedLine as text
%         str2double(get(hObject,'String')) returns contents of CirLFeedLine
as a double


% --- Executes during object creation, after setting all properties.
function CirImpFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirImpFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

```

```

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function CirImpFeedLine_Callback(hObject, eventdata, handles)
% hObject    handle to CirImpFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of CirImpFeedLine as text
%        str2double(get(hObject,'String')) returns contents of
CirImpFeedLine as a double


% --- Executes during object creation, after setting all properties.
function CirWFeedLine_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirWFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end


function CirWFeedLine_Callback(hObject, eventdata, handles)
% hObject    handle to CirWFeedLine (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of CirWFeedLine as text
%        str2double(get(hObject,'String')) returns contents of CirWFeedLine
as a double


% --- Executes during object creation, after setting all properties.
function Rec2PatchFreqGoal_CreateFcn(hObject, eventdata, handles)
% hObject    handle to Rec2PatchFreqGoal (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called

% Hint: edit controls usually have a white background on Windows.
%       See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

```

```
set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end
```

```
function Rec2PatchFreqGoal_Callback(hObject, eventdata, handles)
% hObject    handle to Rec2PatchFreqGoal (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of Rec2PatchFreqGoal as
text
%          str2double(get(hObject,'String')) returns contents of
Rec2PatchFreqGoal as a double
```

```
% --- Executes during object creation, after setting all properties.
function CirPermattivity_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirPermattivity (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called
```

```
% Hint: edit controls usually have a white background on Windows.
%          See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end
```

```
function CirPermattivity_Callback(hObject, eventdata, handles)
% hObject    handle to CirPermattivity (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of CirPermattivity as text
%          str2double(get(hObject,'String')) returns contents of
CirPermattivity as a double
```

```
% --- Executes during object creation, after setting all properties.
function CirFeedPoint_CreateFcn(hObject, eventdata, handles)
% hObject    handle to CirFeedPoint (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    empty - handles not created until after all CreateFcns called
```

```
% Hint: edit controls usually have a white background on Windows.
%          See ISPC and COMPUTER.
if ispc
    set(hObject,'BackgroundColor','white');
else

set(hObject,'BackgroundColor',get(0,'defaultUicontrolBackgroundColor'));
end
```

```

function CirFeedPoint_Callback(hObject, eventdata, handles)
% hObject    handle to CirFeedPoint (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hints: get(hObject,'String') returns contents of CirFeedPoint as text
%        str2double(get(hObject,'String')) returns contents of CirFeedPoint
%        as a double

% --- Executes on button press in RadioRec2.
function RadioRec2_Callback(hObject, eventdata, handles)
% hObject    handle to RadioRec2 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hint: get(hObject,'Value') returns toggle state of RadioRec2

% --- Executes on button press in RadioRec4.
function RadioRec4_Callback(hObject, eventdata, handles)
% hObject    handle to RadioRec4 (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hint: get(hObject,'Value') returns toggle state of RadioRec4

% --- Executes on button press in RadioCir.
function RadioCir_Callback(hObject, eventdata, handles)
% hObject    handle to RadioCir (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)

% Hint: get(hObject,'Value') returns toggle state of RadioCir

```

```

function A = Rectangular2_patch_antenna(h,Er,Fresgoal)

%WPatch Width patchantenn
%h Height substrat
%Er dielectric constant patchantenn
%format long;
%figure;

% Konstanter
Sigma=5.8e7;
Eo=8.85418e-12;
Myo=(4e-7)*pi;
c=2.99792e8;
temp=1e500;

WPatch = (1/(2*Fresgoal*(Myo*Eo)^0.5))*(2/(Er+1))^0.5;
EeffPatch=((Er+1)/2)+((Er-1)/2)*(1+(10*h/WPatch))^0.5;
deltaL=0.412*h*((EeffPatch+.3)/(EeffPatch-0.258))*(((WPatch/h)+.264)/((WPatch/h)+0.813));
Lamdag=(c/(Fresgoal*(Er)^0.5));

%Beräkning av halvvåg
Lamda2=0.48:0.000005:.49;
Fre=zeros(length(Lamda2));
Lt=zeros(length(Lamda2));

for i=1:length(Lamda2)
    Lt=Lamda2*Lamdag;
    Fre=1/((2*(Myo*Eo)^0.5)*(Lt(i)+2*deltaL)*(EeffPatch)^0.5)-Fresgoal;
    Fresp=1/((2*(Myo*Eo)^0.5)*(Lt(i)+2*deltaL)*(EeffPatch)^0.5);
    if abs(real(Fre)) < temp
        temp = abs(real(Fre));
        LPatch = real(Lt(i));
        FresPatch = abs(real(Fresp));
    end
end
LPatch;
FresPatch;

%Microstrip line
%WStrip Width Microstrip line
%Konstanter
Zo=50;
H=h*1000;
Zf=(Myo/Eo)^0.5;
FactorA=2*pi*(Zo/Zf)*((Er+1)/2)^0.5+((Er-1)/(Er+1))*(0.23+(.11/Er));
WStrip=((h*8*exp(real(FactorA)))/(exp(2*real(FactorA))-2));

if (WStrip/h) > 2
    B=((Zf*pi)/(2*Zo*(Er)^0.5));
    WStrip=((2*h)/pi)*(B-1-(log(2*B)-1)/log(exp(1)))+(((Er-1)/(2*Er))*((log(B-1)/log(exp(1)))+.39-(.61/Er)));
end

EeffStrip=((Er+1)/2)+((Er-1)/2)*(1+(12*h/WStrip))^0.5;
LStrip=real((c/(((EeffStrip)^0.5)*FresPatch)));
ZoCheckA=real((Zf/(((EeffStrip)^0.5)*(1.393+(WStrip/h)+((2/3)*(log((WStrip/h)+1.444)/log(exp(1)))))))));

if (WStrip/h) < 1

```



```

    EeffStrip=((Er+1)/2)+((Er-1)/2)*(1+(12*(h/WStrip))^(-0.5)+(.04*(1-
(WStrip/h)^2)));
    LStrip=real((c/(((EeffStrip)^0.5)*FresPatch)));

ZoCheckB=real((Zf*((log(8*(h/WStrip)+(WStrip/(4*h))))/(log(exp(1)))))/((2*pi
i*((EeffStrip)^0.5))));
    ZoCheckA = ZoCheckB;
end

%Impedans
ko=(2*pi)/(c/Fresgoal);
Wb=35e-3*H*.18;
Y=ko*LPatch;
Lamdao = c/Fresgoal;

I2=(1/(120*pi^2))*((-2*Y-Y*cos(Y)-sin(Y)-(Y)^2*sinint(Y))/(2*Y))+(-
2*Y+Y*cos(Y)+sin(Y)+(Y)^2*sinint(Y))/(2*Y));
G1=((pi*WPatch)/(Lamdao*Zf))*(1+(1-(0.636*(log(ko*Wb)/log(exp(1))))));
G12=I2*besselj(0,2*Y);

%Edge Impedance
EdgeRin=real((((G1+G12))^-1));

%Impedance with Xo
xo=0.0:0.000005:LPatch;
RinDeltaXo=zeros(length(xo));
Rin=zeros(length(xo));
for i=1:length(xo)
    RinDeltaXo=((2*(G1+G12))^-1*(cos((pi*xo(i))/LPatch)))-Zo;
    Rin=((2*(G1+G12))^-1*(cos((pi*xo(i))/LPatch)));
    if abs(RinDeltaXo) < temp
        temp = abs(RinDeltaXo);
        Xo=xo(i);
        RinXo = Rin;
    end
end

Lamda = c/Fresgoal;
%Radiation pattern
Degree=0:(2*pi/360):2*pi;
%E-plane
Fe=((sin((ko*h/2)*cos(Degree)))/((ko*h/2)*cos(Degree)))*(cos((ko*h/2)*cos(
Degree)));
%polar(Degree, Fe, '-r')
%hold on
%F-plane
Ff=((sin((ko*WPatch/2)*cos(Degree)))/((ko*WPatch/2)*cos(Degree)))*(sin(Deg
ree));
%polar(Degree, Ff, '-b')
A=[Sigma, Eo, Myo, c, Zo, EeffPatch, deltaL, LPatch, FresPatch, FactorA, WStrip, LStrip,
EeffStrip, ZoCheckA, ko, EdgeRin, Xo, RinXo, Lamda, real(WPatch*1000)]; %20st

```

```

function A = Rectangular4_patch_antenna(h,Er,Fresgoal)

%WPatch Width patchantenn
%h Height substrat
%Er dielectric constant patchantenn

% Konstanter
Sigma=5.8e7;
Eo=8.85418e-12;
Myo=(4e-7)*pi;
c=2.99792e8;
temp=1e500;

WPatch = (1/(2*Fresgoal*(Myo*Eo)^0.5))*(2/(Er+1))^0.5;

EeffPatch=((Er+1)/2)+((Er-1)/2)*(1+(10*h/WPatch))^-0.5;
deltaL=0.412*h*((EeffPatch+.3)/(EeffPatch-0.258))*((WPatch/h)+.264)/((WPatch/h)+0.813));
Lamdag=(c/(Fresgoal*(Er)^0.5));

%Beräkning av Quater wave
Lamda2=0.48:0.000005:.49;
Fre=zeros(length(Lamda2));
Lt=zeros(length(Lamda2));

for i=1:length(Lamda2)
    Lt=Lamda2*Lamdag;
    Fre=1/((2*(Myo*Eo)^0.5)*(Lt(i)+2*deltaL)*(EeffPatch)^0.5)-Fresgoal;
    Fresp=1/((2*(Myo*Eo)^0.5)*(Lt(i)+2*deltaL)*(EeffPatch)^0.5);
    if abs(real(Fre)) < temp
        temp = abs(real(Fre));
        LPatch = real(Lt(i));
        FresPatch = abs(real(Fresp));
    end
end

%Microstrip line
%WStrip Width Microstrip line
%Konstanter
Zo=50;
H=h*1000;
Zf=(Myo/Eo)^0.5;
FactorA=2*pi*(Zo/Zf)*((Er+1)/2)^0.5+((Er-1)/(Er+1))*(0.23+(.11/Er));
WStrip=((h*8*exp(real(FactorA)))/(exp(2*real(FactorA))-2));

if (WStrip/h) > 2
    B=((Zf*pi)/(2*Zo*(Er)^0.5));
    WStrip=((2*h)/pi)*(B-1-(log(2*B)-1)/log(exp(1)))+(((Er-1)/(2*Er))*((log(B-1)/log(exp(1)))+.39-(.61/Er)));
end

EeffStrip=((Er+1)/2)+((Er-1)/2)*(1+(12*h/WStrip))^-0.5;
LStrip=real((c/(((EeffStrip)^0.5)*FresPatch)));
ZoCheckA=real((Zf/(((EeffStrip)^0.5)*(1.393+(WStrip/h)+((2/3)*(log((WStrip/h)+1.444)/log(exp(1)))))))));

if (WStrip/h) < 1
    EeffStrip=((Er+1)/2)+((Er-1)/2)*(1+(12*(h/WStrip))^-0.5+(.04*(1-(WStrip/h))^2));
    LStrip=real((c/(((EeffStrip)^0.5)*FresPatch)));
end

```

```

ZoCheckB=real((Zf*((log(8*(h/WStrip)+(WStrip/(4*h))))/(log(exp(1)))))/((2*pi*
i*((EeffStrip)^0.5))));
ZoCheckA = ZoCheckB;
end

%Impedans
LPatch2 = LPatch/2;
ko=(2*pi)/(c/Fresgoal);
Wb=(35e-3*H*0.8);
Y=ko*LPatch2;
Lamdao = c/Fresgoal;

I2=(1/(120*pi^2))*((-2*Y-Y*cos(Y)-sin(Y)-(Y)^2*sinint(Y))/(2*Y))+(-
2*Y+Y*cos(Y)+sin(Y)+(Y)^2*sinint(Y))/(2*Y));
G1=((pi*WPatch)/(Lamdao*Zf))*(1+(1-(0.636*(log(ko*Wb)/log(exp(1))))));
G12=I2*besselj(0,2*Y);

%Edge Impedance
EdgeRin=real((((G1+G12))^-1));

%Impedance with Xo
xo=0:0.000005:LPatch;
RinDeltaXo=zeros(length(xo));
Rin=zeros(length(xo));
for i=1:length(xo)
    RinDeltaXo=((2*(G1+G12))^-1*(cos((pi*xo(i))/LPatch)))-Zo;
    Rin=((2*(G1+G12))^-1*(cos((pi*xo(i))/LPatch)));
    if abs(RinDeltaXo) < temp
        temp = abs(RinDeltaXo);
        Xo=xo(i);
        RinXo = Rin;
    end
end
end
Lamda = c/Fresgoal;
LPatch=LPatch/2;

%Radiation pattern
Degree=0:(2*pi/360):2*pi;

%E-plane
Fe=((sin((ko*h/2)*cos(Degree)))/((ko*h/2)*cos(Degree)))*(cos((ko*h/2)*cos(
Degree)));
%polar(Degree, Fe, '--r')
%hold on
%F-plane
Ff=((sin((ko*WPatch/2)*cos(Degree)))/((ko*WPatch/2)*cos(Degree)))*(sin(Deg
ree));
%polar(Degree, Ff, '--b')
A=[Sigma, Eo, Myo, c, Zo, EeffPatch, deltaL, LPatch, FresPatch, FactorA, WStrip, LStri
p, EeffStrip, ZoCheckA, ko, EdgeRin, Xo, RinXo, Lamda, real(WPatch*1000)];

```

```

function A = Rectangular4_patch_antenna(h,Er,Fresgoal)

%WPatch Width patchantenn
%h Height substrat
%Er dielectric constant patchantenn

% Konstanter
Sigma=5.8e7;
Eo=8.85418e-12;
Myo=(4e-7)*pi;
c=2.99792e8;
temp=1e500;

WPatch = (1/(2*Fresgoal*(Myo*Eo)^0.5))*(2/(Er+1))^0.5;

EeffPatch=((Er+1)/2)+((Er-1)/2)*(1+(10*h/WPatch))^-0.5;
deltaL=0.412*h*((EeffPatch+.3)/(EeffPatch-0.258))*((WPatch/h)+.264)/((WPatch/h)+0.813));
Lamdag=(c/(Fresgoal*(Er)^0.5));

%Beräkning av Quater wave
Lamda2=0.48:0.000005:.49;
Fre=zeros(length(Lamda2));
Lt=zeros(length(Lamda2));

for i=1:length(Lamda2)
    Lt=Lamda2*Lamdag;
    Fre=1/((2*(Myo*Eo)^0.5)*(Lt(i)+2*deltaL)*(EeffPatch)^0.5)-Fresgoal;
    Fresp=1/((2*(Myo*Eo)^0.5)*(Lt(i)+2*deltaL)*(EeffPatch)^0.5);
    if abs(real(Fre)) < temp
        temp = abs(real(Fre));
        LPatch = real(Lt(i));
        FresPatch = abs(real(Fresp));
    end
end

%Microstrip line
%WStrip Width Microstrip line
%Konstanter
Zo=50;
H=h*1000;
Zf=(Myo/Eo)^0.5;
FactorA=2*pi*(Zo/Zf)*((Er+1)/2)^0.5+((Er-1)/(Er+1))*(0.23+(.11/Er));
WStrip=((h*8*exp(real(FactorA)))/(exp(2*real(FactorA))-2));

if (WStrip/h) > 2
    B=((Zf*pi)/(2*Zo*(Er)^0.5));
    WStrip=((2*h)/pi)*(B-1-(log(2*B)-1)/log(exp(1)))+(((Er-1)/(2*Er))*((log(B-1)/log(exp(1)))+.39-(.61/Er)));
end

EeffStrip=((Er+1)/2)+((Er-1)/2)*(1+(12*h/WStrip))^-0.5;
LStrip=real((c/(((EeffStrip)^0.5)*FresPatch)));
ZoCheckA=real((Zf/(((EeffStrip)^0.5)*(1.393+(WStrip/h)+((2/3)*(log((WStrip/h)+1.444)/log(exp(1)))))))));

if (WStrip/h) < 1
    EeffStrip=((Er+1)/2)+((Er-1)/2)*(1+(12*(h/WStrip))^-0.5+(.04*(1-(WStrip/h))^2));
    LStrip=real((c/(((EeffStrip)^0.5)*FresPatch)));
end

```

```

ZoCheckB=real((Zf*((log(8*(h/WStrip)+(WStrip/(4*h))))/(log(exp(1)))))/((2*pi*
i*((EeffStrip)^0.5))));
ZoCheckA = ZoCheckB;
end

%Impedans
LPatch2 = LPatch/2;
ko=(2*pi)/(c/Fresgoal);
Wb=(35e-3*H*0.8);
Y=ko*LPatch2;
Lamdao = c/Fresgoal;

I2=(1/(120*pi^2))*((-2*Y-Y*cos(Y)-sin(Y)-(Y)^2*sinint(Y))/(2*Y))+(-
2*Y+Y*cos(Y)+sin(Y)+(Y)^2*sinint(Y))/(2*Y));
G1=((pi*WPatch)/(Lamdao*Zf))*(1+(1-(0.636*(log(ko*Wb)/log(exp(1))))));
G12=I2*besselj(0,2*Y);

%Edge Impedance
EdgeRin=real((((G1+G12))^-1));

%Impedance with Xo
xo=0:0.000005:LPatch;
RinDeltaXo=zeros(length(xo));
Rin=zeros(length(xo));
for i=1:length(xo)
    RinDeltaXo=((2*(G1+G12))^-1*(cos((pi*xo(i))/LPatch)))-Zo;
    Rin=((2*(G1+G12))^-1*(cos((pi*xo(i))/LPatch)));
    if abs(RinDeltaXo) < temp
        temp = abs(RinDeltaXo);
        Xo=xo(i);
        RinXo = Rin;
    end
end
end
Lamda = c/Fresgoal;
LPatch=LPatch/2;

%Radiation pattern
Degree=0:(2*pi/360):2*pi;

%E-plane
Fe=((sin((ko*h/2)*cos(Degree)))/((ko*h/2)*cos(Degree)))*(cos((ko*h/2)*cos(
Degree)));
%polar(Degree, Fe, '--r')
%hold on
%F-plane
Ff=((sin((ko*WPatch/2)*cos(Degree)))/((ko*WPatch/2)*cos(Degree)))*(sin(Deg
ree));
%polar(Degree, Ff, '--b')
A=[Sigma, Eo, Myo, c, Zo, EeffPatch, deltaL, LPatch, FresPatch, FactorA, WStrip, LStri
p, EeffStrip, ZoCheckA, ko, EdgeRin, Xo, RinXo, Lamda, real(WPatch*1000)];

```